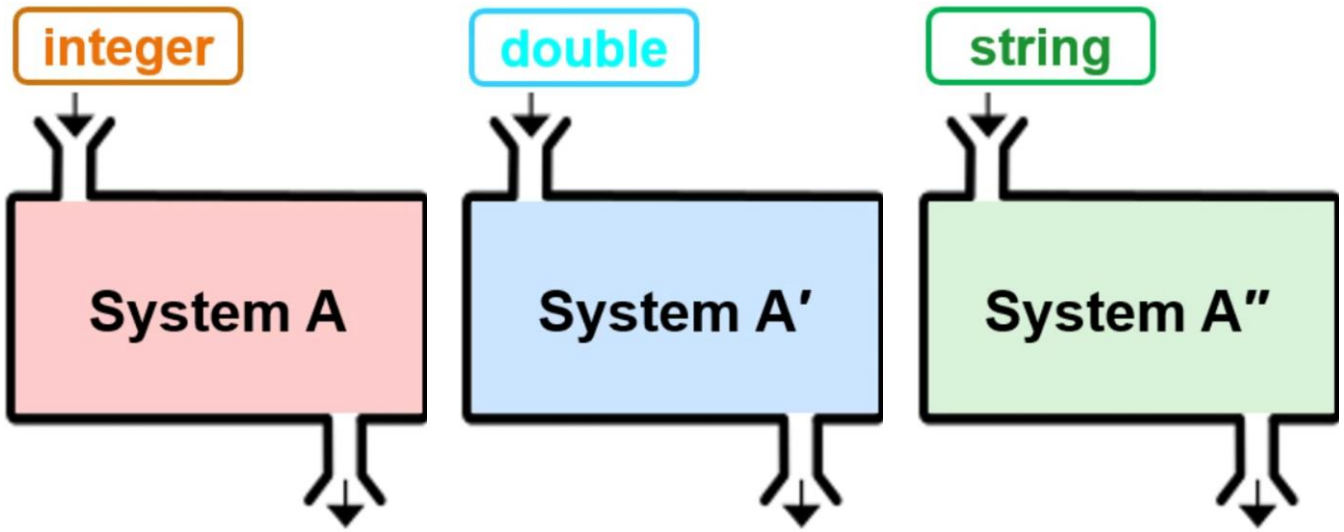
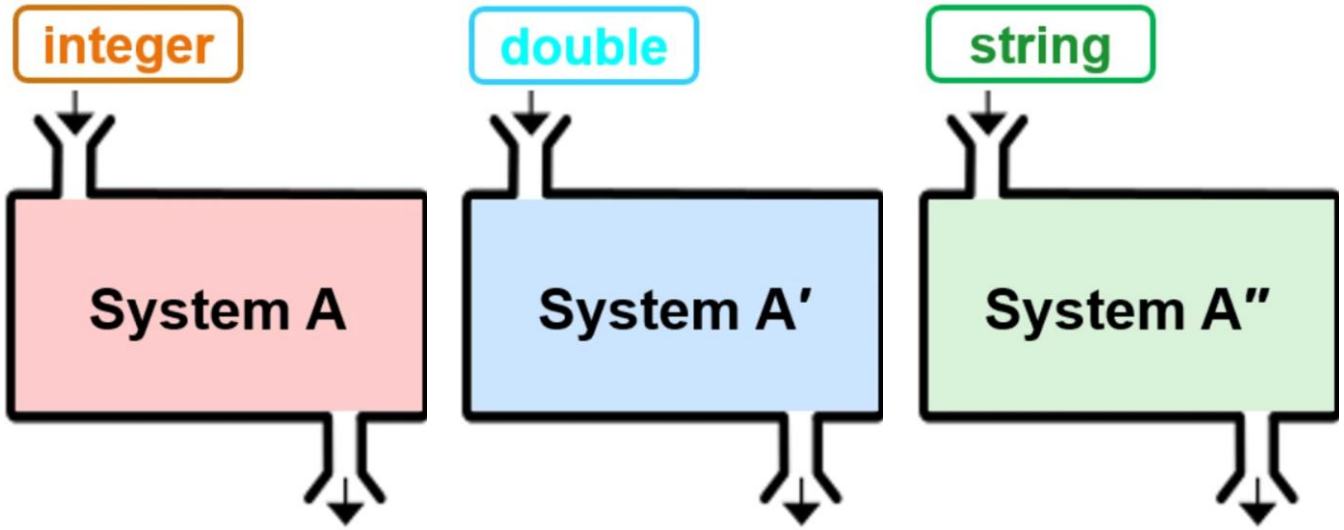
Lesson 10: Chapter19

Generics

Motivations



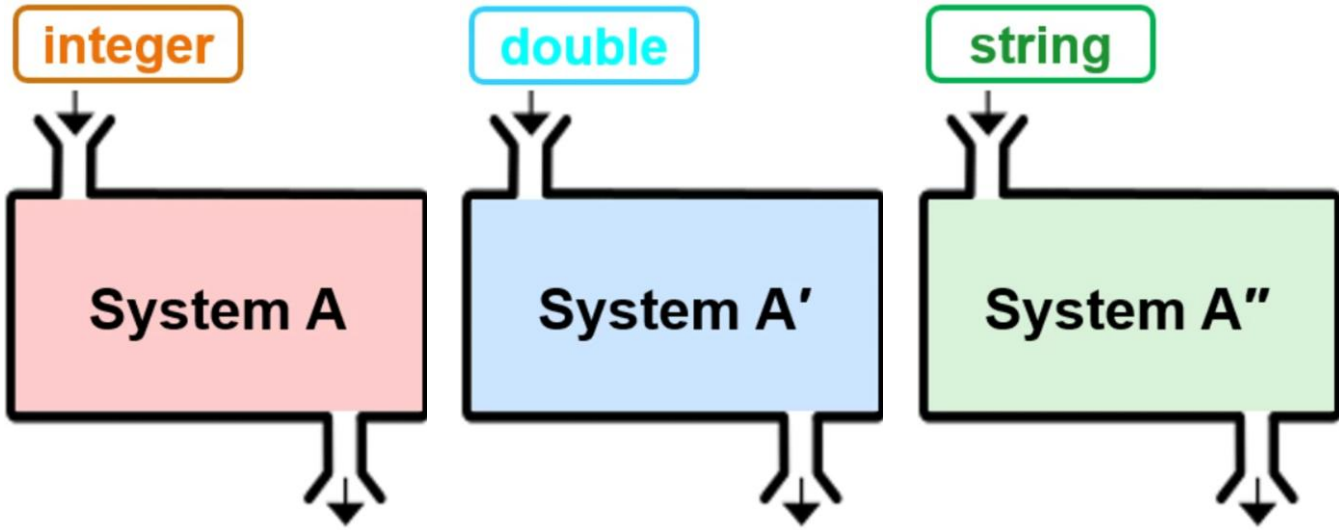
Motivations



The Same Logic, Different Data Types:

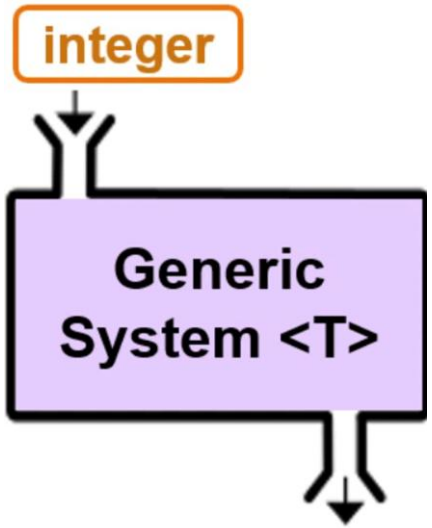
- **“Type-Specific” Code**
- **Code duplication**
- **Increased maintenance burden**

Motivations



The Same Logic, Different Data Types:

- "Type-Specific" Code
- Code duplication
- Increased maintenance burden



Generics

One Logic for All Types:

- "Solve Once, Use Everywhere"
- No code duplication
- Reduces maintenance effort

Objectives

19.1 To know the benefits of generics (§19.1).

19.2 To use generic classes and interfaces (§19.2).

19.3 To declare generic classes and interfaces (§19.3).

19.4 To understand why generic types can improve reliability and readability (§19.3).

19.5 To declare and use generic methods and bounded generic types (§19.4).

19.6 To use raw types for backward compatibility (§19.5).

19.7 To know wildcard types and understand why they are necessary (§19.6).

19.8 To convert legacy code using JDK 1.5 generics (§19.7).

19.9 To understand that generic type information is erased by the compiler and all instances of a generic class share the same runtime class file (§19.8).

19.10 To know certain restrictions on generic types caused by type erasure (§19.8).

19.11 To design and implement generic matrix classes (§19.9).

Objectives

19.1 To know the benefits of **generics** (§19.1).

19.2 To use **generic classes** and **interfaces** (§19.2).

19.3 To declare generic classes and interfaces (§19.3).

19.4 To understand why generic types can improve **reliability** and **readability** (§19.3).

19.5 To declare and use **generic methods** and **bounded generic types** (§19.4).

19.6 To use raw types for backward compatibility (§19.5).

19.7 To know **wildcard types** and understand why they are necessary (§19.6).

19.8 To convert legacy code using JDK 1.5 generics (§19.7).

19.9 To understand that generic type information is erased by the compiler and all instances of a generic class share the same runtime class file (§19.8).

19.10 To know certain restrictions on generic types caused by type erasure (§19.8).

19.11 To design and implement **generic matrix classes** (§19.9).

ge·ner·ic /dʒəˈnerɪk/ ●○○ **adjective** [**usually before noun**]

ge·ner·ic /dʒəˈnerɪk/ ●○○ **adjective** [**usually before noun**]

ORIGIN OF GENERIC[1]

First recorded in 1670–80; from Latin *gener-* (see gender[1]) + -ic

ORIGIN OF GENDER[1]

First recorded in 1300–50; Middle English, from Middle French *gendre*, *genre*, from Latin *gener-* (stem of *genus*) “kind, sort”

ge·ner·ic /dʒəˈnerɪk/ ●○○ **adjective** [**usually before noun**]

1 relating to a whole group of things rather than to one thing

generic term/name (for something)

Fine Arts is a generic term for subjects such as painting, music, and sculpture.

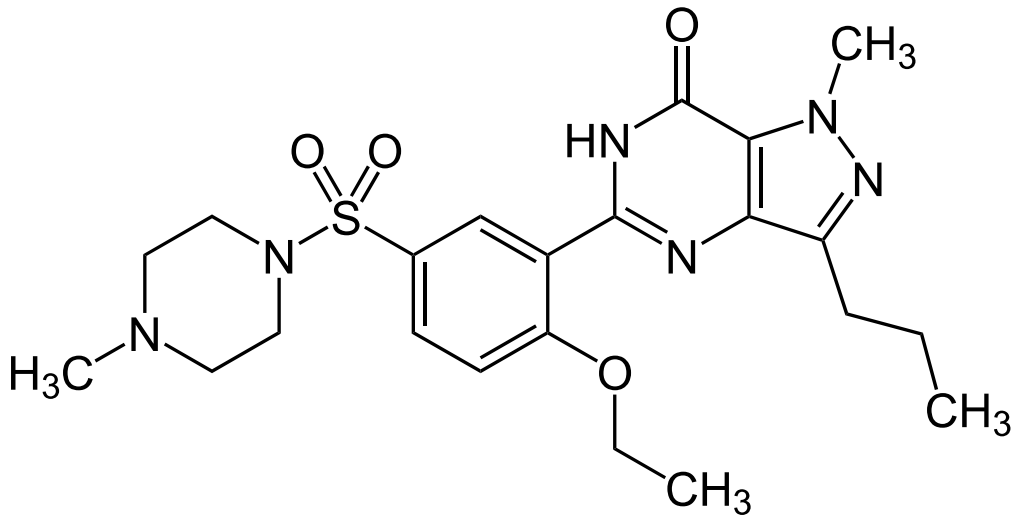
2 a generic product does not have a special name to show that it is made by a particular company

generic drugs

Generic drugs

Brand name

Active Ingredient



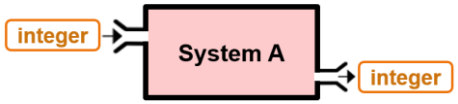
Phosphodiesterase 5 inhibitor (PDE5 inhibitor)

Generic drug

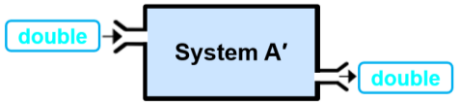
~ $90

~ $5

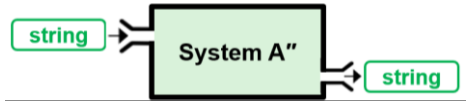
Why Generics?




```
class IntegerSys {
  private int value;
  public IntegerSys(int value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

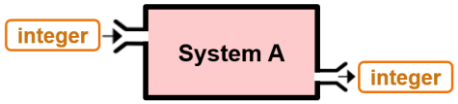



```
class DoubleSys {
  private double value;
  public DoubleSys(double value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

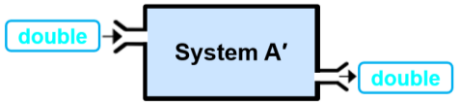



```
class StringSys {
  private String value;
  public StringSys(String value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

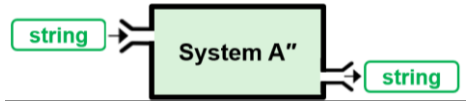
Why Generics?




```java
class IntegerSys {
  private int value;
  public IntegerSys(int value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```



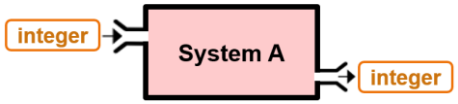

```java
class DoubleSys {
  private double value;
  public DoubleSys(double value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```



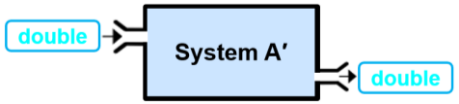

```java
class StringSys {
  private String value;
  public StringSys(String value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

```java
// Without Generics
IntegerSys intSysOld    = new IntegerSys(100);
DoubleSys doubleSysOld = new DoubleSys(99.99);
StringSys stringSysOld = new StringSys("Redundant");
```

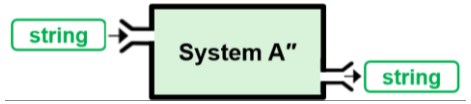
Why Generics?




```java
class IntegerSys {
  private int value;
  public IntegerSys(int value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```



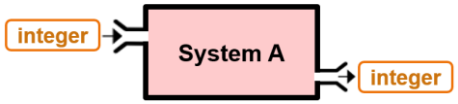

```java
class DoubleSys {
  private double value;
  public DoubleSys(double value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```



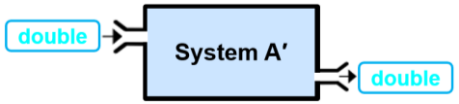

```java
class StringSys {
  private String value;
  public StringSys(String value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

```java
// Without Generics
IntegerSys intSysOld    = new IntegerSys(100);
DoubleSys doubleSysOld = new DoubleSys(99.99);
StringSys stringSysOld = new StringSys("Redundant");

intSysOld.display();    ?
```

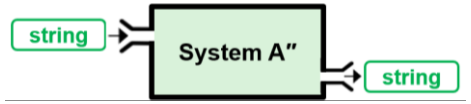
Why Generics?




```java
class IntegerSys {
  private int value;
  public IntegerSys(int value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```



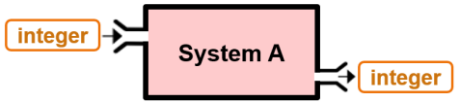

```java
class DoubleSys {
  private double value;
  public DoubleSys(double value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```



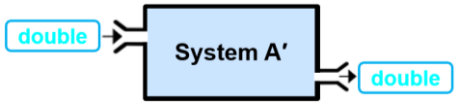

```java
class StringSys {
  private String value;
  public StringSys(String value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

```java
// Without Generics
IntegerSys intSysOld    = new IntegerSys(100);
DoubleSys doubleSysOld = new DoubleSys(99.99);
StringSys stringSysOld = new StringSys("Redundant");

intSysOld.display();          100
doubleSysOld.display(); ?
```

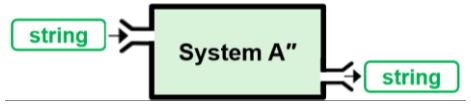
Why Generics?




```java
class IntegerSys {
  private int value;
  public IntegerSys(int value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```




```java
class DoubleSys {
  private double value;
  public DoubleSys(double value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

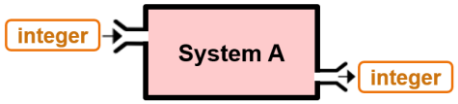



```java
class StringSys {
  private String value;
  public StringSys(String value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

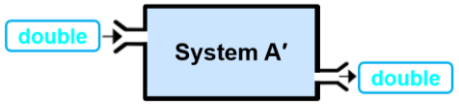
```java
// Without Generics
IntegerSys intSysOld    = new IntegerSys(100);
DoubleSys doubleSysOld = new DoubleSys(99.99);
StringSys stringSysOld = new StringSys("Redundant");

intSysOld.display();          100
doubleSysOld.display();       99.99
stringSysOld.display();
```

?

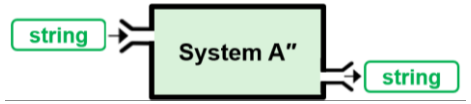
Why Generics?




```java
class IntegerSys {
  private int value;
  public IntegerSys(int value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```



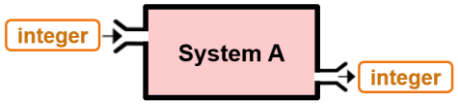

```java
class DoubleSys {
  private double value;
  public DoubleSys(double value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```



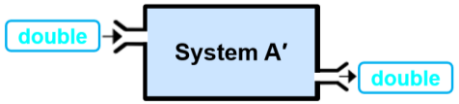

```java
class StringSys {
  private String value;
  public StringSys(String value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

```java
// Without Generics
IntegerSys intSysOld    = new IntegerSys(100);
DoubleSys doubleSysOld = new DoubleSys(99.99);
StringSys stringSysOld = new StringSys("Redundant");

intSysOld.display();          100
doubleSysOld.display();       99.99
stringSysOld.display();       Redundant
```

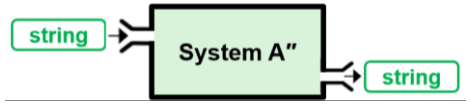
Why Generics?




```java
class IntegerSys {
  private int value;
  public IntegerSys(int value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```




```java
class DoubleSys {
  private double value;
  public DoubleSys(double value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```




```java
class StringSys {
  private String value;
  public StringSys(String value){
      this.value = value;
  }
  public void display(){
      System.out.println(value);
  }
}
```

```java
// Without Generics
IntegerSys intSysOld    = new IntegerSys(100);
DoubleSys doubleSysOld = new DoubleSys(99.99);
StringSys stringSysOld = new StringSys("Redundant");

intSysOld.display();          100
doubleSysOld.display();       99.99
stringSysOld.display();       Redundant
```

The Same Logic, Different Data Types:

- "Type-Specific" Code
- Code duplication
- Increased maintenance burden

Why Generics?

```
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

Why Generics?

Generic class declaration (signature)

```
class GenericSys<T> {
    private   value;
    public GenericSys(  value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

Why Generics?

Type parameter *placeholder*:
Defines a **type**
that can be replaced with **any data type**

```
class GenericSys<T> {
    private   value;
    public GenericSys(  value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

Why Generics?

Type parameter *placeholder*:
Defines a **type**
that can be replaced with **any data type**

Declare **variables** using
the **generic type** as needed

```
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

Why Generics?

Type parameter *placeholder*:
Defines a **type**
that can be replaced with **any data type**

Declare **variables** using
the **generic type** as needed

```
class GenericSys<E> {
    private E value;
    public GenericSys(E value) {
        this.value = value;
    }
    public E getValue() {
        return value;
    }
}
```

Use *any letter* – just stay consistent

Type Parameter Conventions	
T	**T**ype
E	**E**lement
K	**K**ey
V	**V**alue
N	**N**umber
S	**S**econdary Type
U	**U**tility or **U**enrelated Type
R	**R**eturn Type

Why Generics?

Type parameter *placeholder*:
Defines a **type**
that can be replaced with **any data type**

Declare **variables** using
the **generic type** as needed

Now variables can hold any type

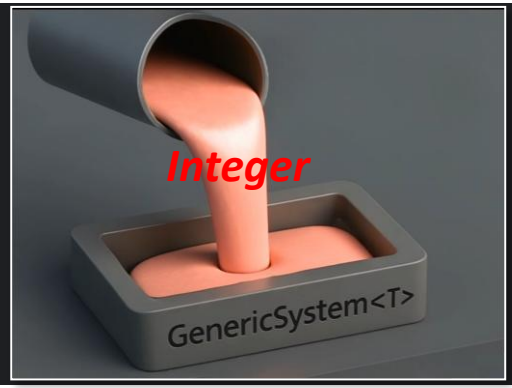
```
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

Why Generics?

```java
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

For `intSys` object, the type `T` inside `GenericSys<T>` should be replaced with ***Integer***

```java
GenericSys<Integer> intSys    = new GenericSys<>(100);
```

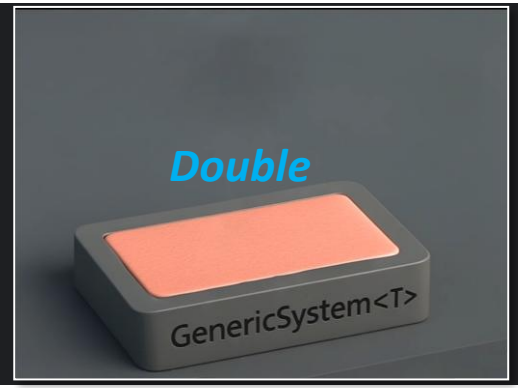


Why Generics?

```java
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

For `doubleSys` object, the type `T` inside `GenericSys<T>` should be replaced with ***Double***

```java
GenericSys<Integer> intSys    = new GenericSys<>(100);
GenericSys<Double> doubleSys = new GenericSys<>(99.99);
```

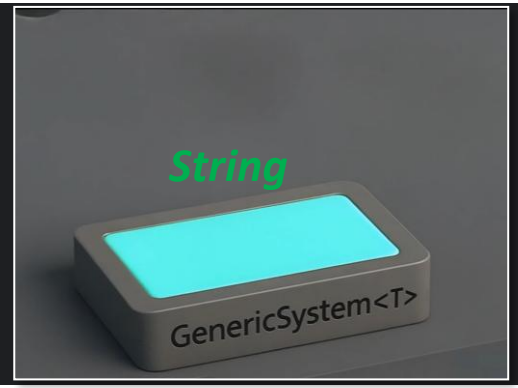


Why Generics?

```java
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

For `stringSys` object, the type `T` inside `GenericSys<T>` should be replaced with *String*

```java
GenericSys<Integer> intSys    = new GenericSys<>(100);
GenericSys<Double> doubleSys = new GenericSys<>(99.99);
GenericSys<String> stringSys = new GenericSys<>("No redundancy");
```



Why Generics?

```java
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

```java
GenericSys<Integer> intSys    = new GenericSys<>(100);
GenericSys<Double> doubleSys = new GenericSys<>(99.99);
GenericSys<String> stringSys = new GenericSys<>("No redundancy");

intSys.display();
```

Why Generics?

```
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

```
GenericSys<Integer> intSys    = new GenericSys<>(100);
GenericSys<Double> doubleSys = new GenericSys<>(99.99);
GenericSys<String> stringSys = new GenericSys<>("No redundancy");

intSys.display();
doubleSys.display();
```

```
100
```

Why Generics?

```
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

```
GenericSys<Integer> intSys    = new GenericSys<>(100);
GenericSys<Double> doubleSys = new GenericSys<>(99.99);
GenericSys<String> stringSys = new GenericSys<>("No redundancy");

intSys.display();              100
doubleSys.display();           99.99
stringSys.display();    ?
```

Why Generics?

```
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

```
GenericSys<Integer> intSys    = new GenericSys<>(100);
GenericSys<Double> doubleSys = new GenericSys<>(99.99);
GenericSys<String> stringSys = new GenericSys<>("No redundancy");

intSys.display();           100
doubleSys.display();        99.99
stringSys.display();        No Redundancy
```

Why Generics?

```java
class GenericSys<T> {
    private T value;
    public GenericSys(T value) {
        this.value = value;
    }
    public void display() {
        System.out.println(value);
    }
}
```

```java
GenericSys<Integer> intSys    = new GenericSys<>(100);
GenericSys<Double> doubleSys = new GenericSys<>(99.99)
GenericSys<String> stringSys = new GenericSys<>("No re

intSys.display();          100
doubleSys.display();       99.99
stringSys.display();       No Redundancy
```

One Logic for All Types:

- "Solve Once, Use Everywhere"
- *No code duplication*
- Reduces maintenance effort
- *Early error detection* at *compile time*

Integer ≠ int

```
GenericSys<int>       →    GenericSys<          >
ArrayList <double>    →    ArrayList <          >
List      <String>
```

Integer ≠ int

```
GenericSys<int>       →    GenericSys<          >
ArrayList <double>    →    A    List <          >
List      <String>
```

Integer ≠ int

Premitive type

```
GenericSys<int>       →    GenericSys<         >
ArrayList <double>    →    ArrayList <         >
List      <String>
```

Integer ≠ int

Premitive type

```
GenericSys<int>       →    GenericSys<Integer>
ArrayList <double>    →    ArrayList <Double>
List      <String>
```

Integer ≠ int

Primitive type

```
GenericSys<int>       →    GenericSys<Integer>
ArrayList <double>    →    ArrayList <Double>
List      <String>
```

Integer ≠ int

```
GenericSys<int>       →    GenericSys<Integer>
ArrayList <double>    →    ArrayList <Double>
List      <String>
```

Reference type: Class

Integer ≠ int

```
GenericSys<int>       →    GenericSys<Integer>
ArrayList <double>    →    ArrayList <Double>
List      <String>
```

Reference type : Wrapper class

Reference type: Class

- **Generics** only work with **reference types**,
  so all **primitive** type should be converted to their corresponding **Wrapper class**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

What is a Wrapper Class?

Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

What is a Wrapper Class?

Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

What is a Wrapper Class?

Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

Wrapper class:
box for primitive values

What is a Wrapper Class?

Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

Wrapper class:
box for primitive values

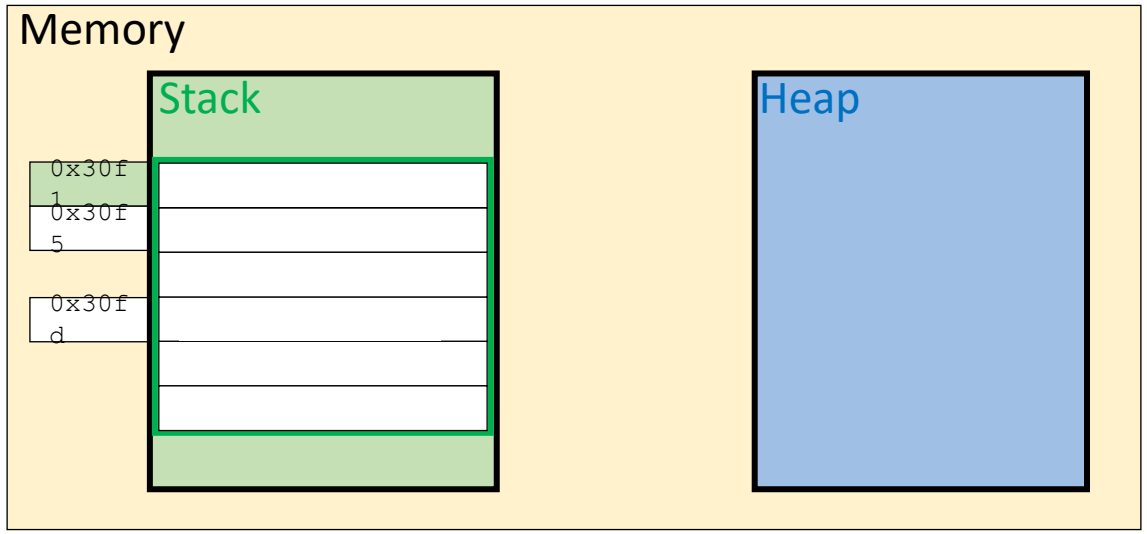
What is a Wrapper Class?

Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character


```
Integer obj = 3;      // Auto-boxing   (int      --> Integer)
```



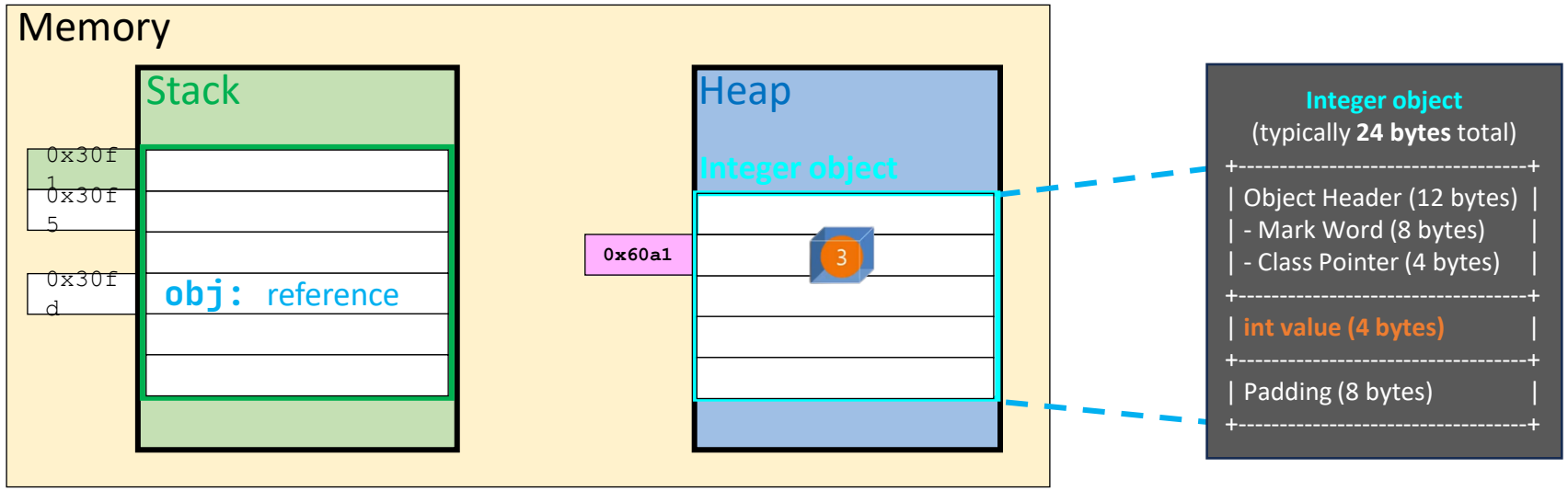
What is a Wrapper Class?

Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

```
Integer obj = 3;      // Auto-boxing   (int      --> Integer)
```



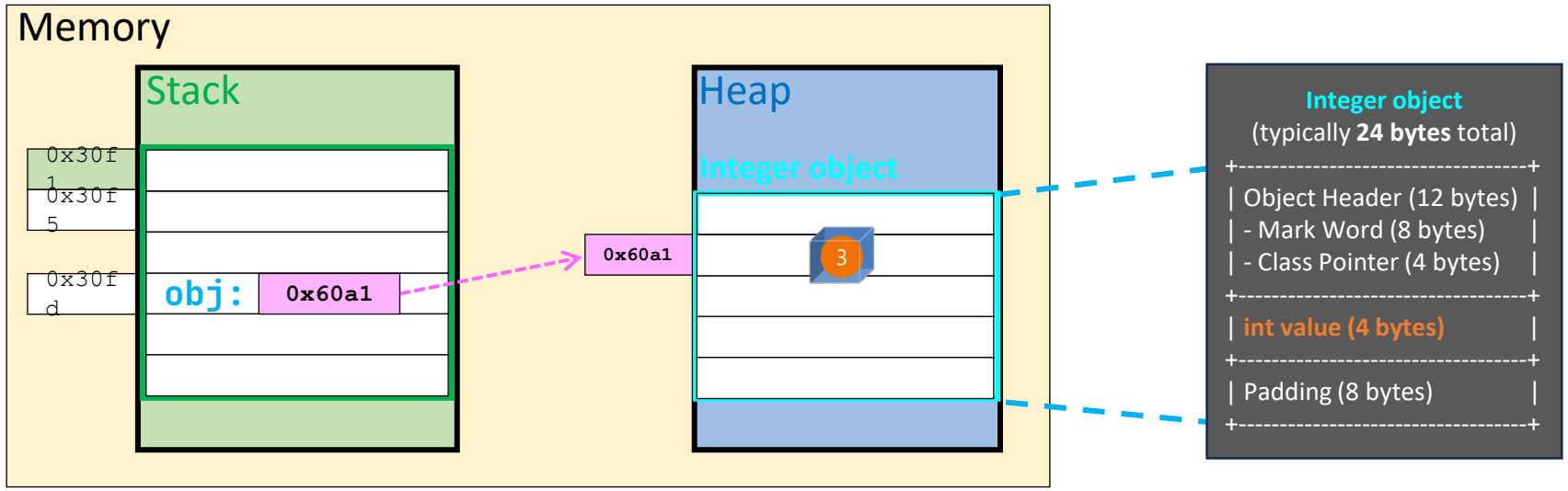
What is a Wrapper Class?

Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

```
Integer obj = 3;      // Auto-boxing   (int      --> Integer)
```



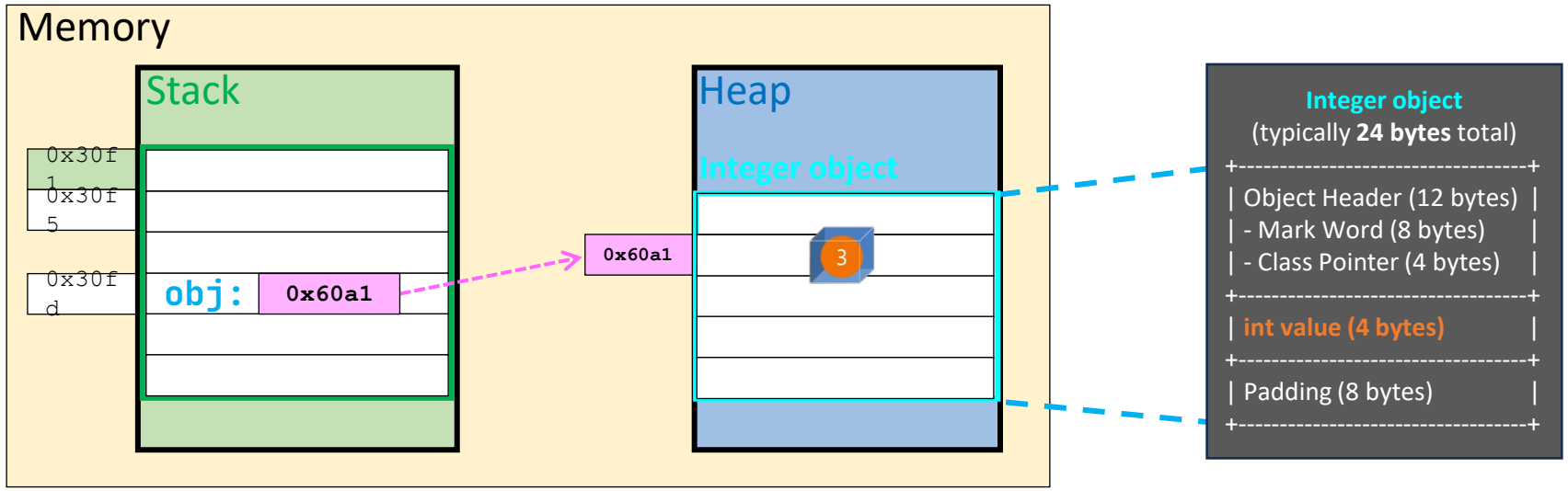
What is a Wrapper Class?

Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

```
Integer obj = 3;      // Auto-boxing   (int     --> Integer)
int     num = obj;    // Auto-unboxing (Integer --> int    )
```



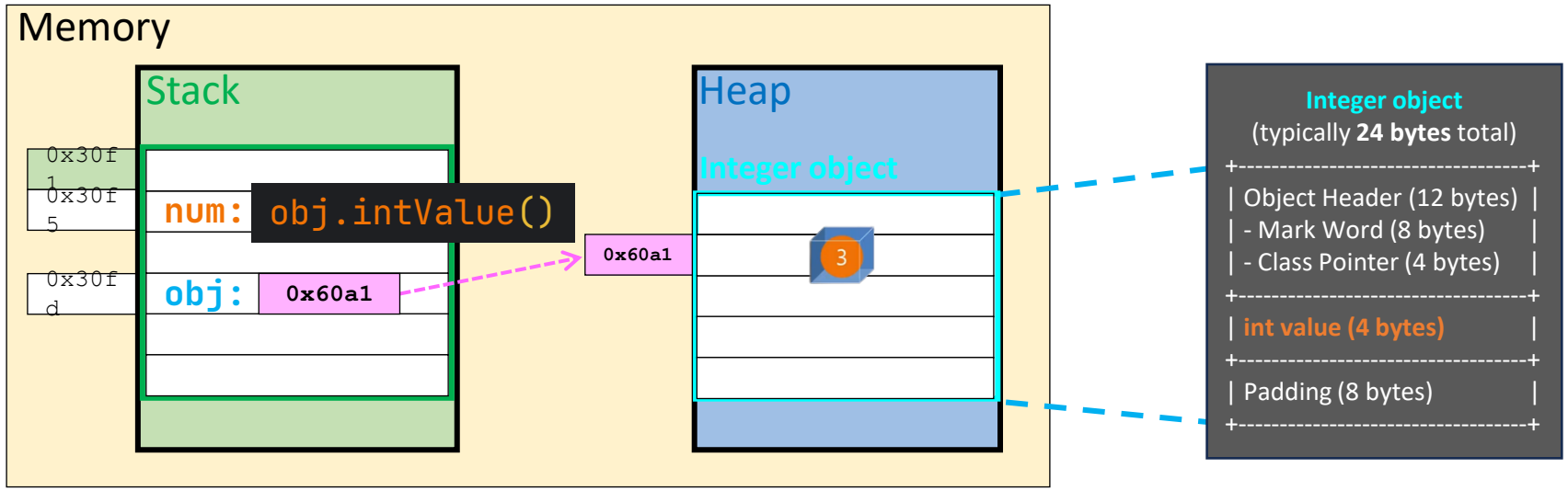
What is a Wrapper Class?

Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

```
Integer obj = 3;      // Auto-boxing   (int     --> Integer)
int     num = obj;    // Auto-unboxing (Integer --> int    )
```



What is a Wrapper Class?

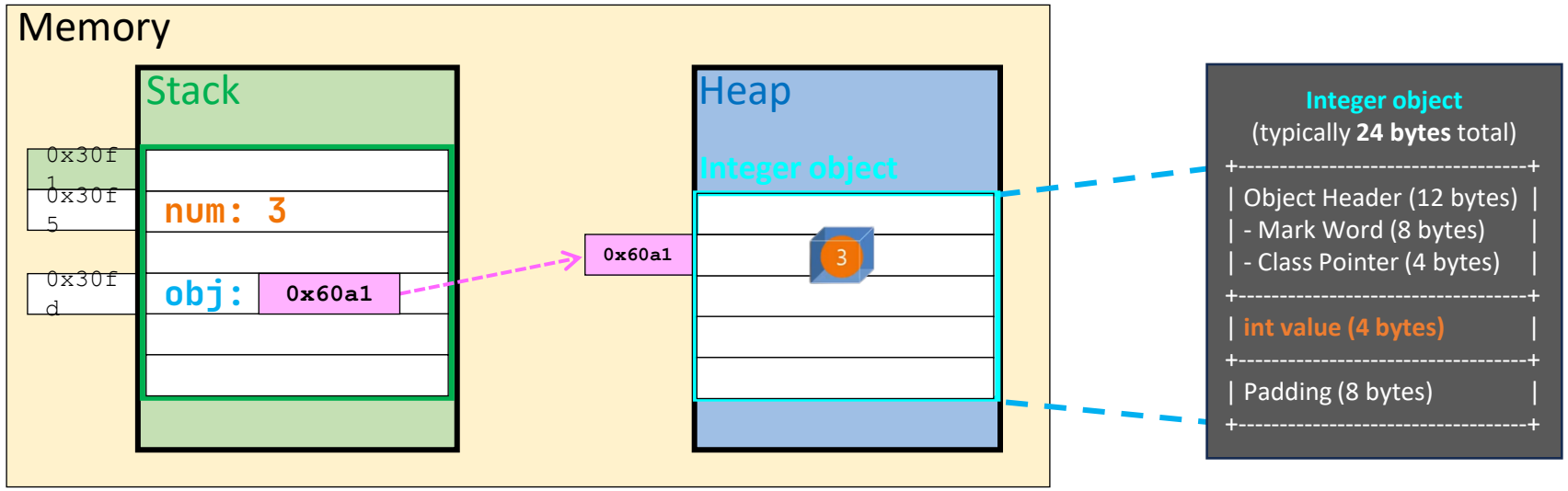
Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

```
Integer obj = 3;      // Auto-boxing   (int     --> Integer)
int     num = obj;    // Auto-unboxing (Integer --> int    )
```

```
After auto-boxing   : obj = ?
After auto-unboxing : num =
```



What is a Wrapper Class?

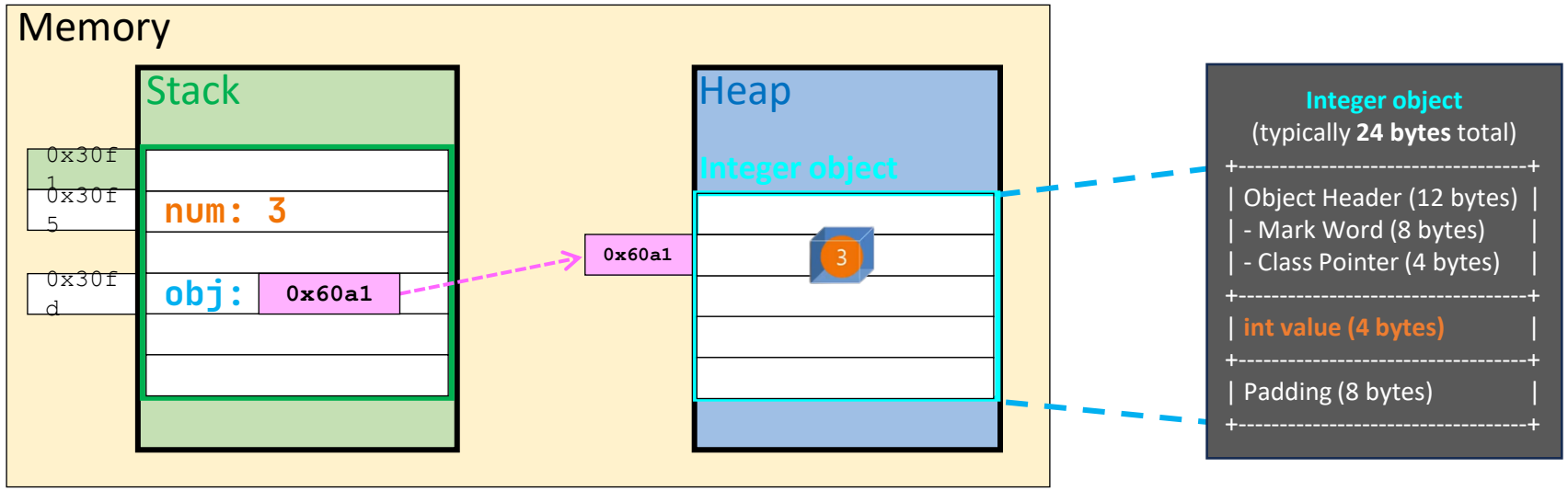
Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

```
Integer obj = 3;      // Auto-boxing   (int     --> Integer)
int     num = obj;    // Auto-unboxing (Integer --> int    )
```

```
After auto-boxing   : obj = obj.toString()
After auto-unboxing : num =
```



What is a Wrapper Class?

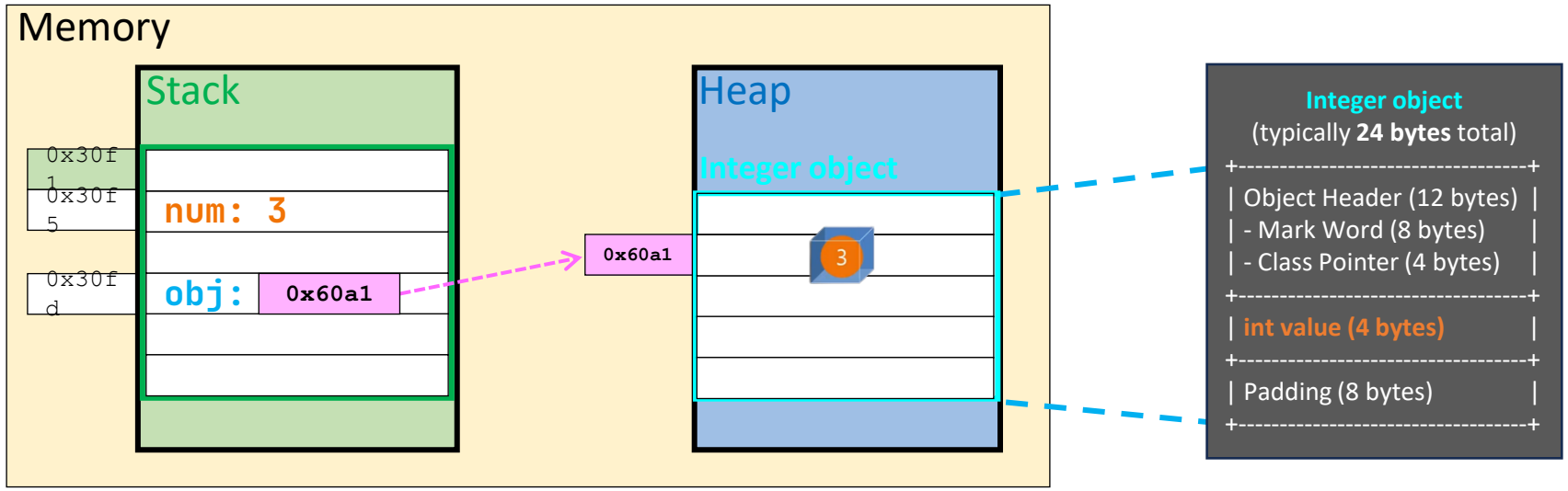
Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

```
Integer obj = 3;      // Auto-boxing   (int     --> Integer)
int     num = obj;    // Auto-unboxing (Integer --> int    )
```

```
After auto-boxing   : obj = 3
After auto-unboxing: num =
```



What is a Wrapper Class?

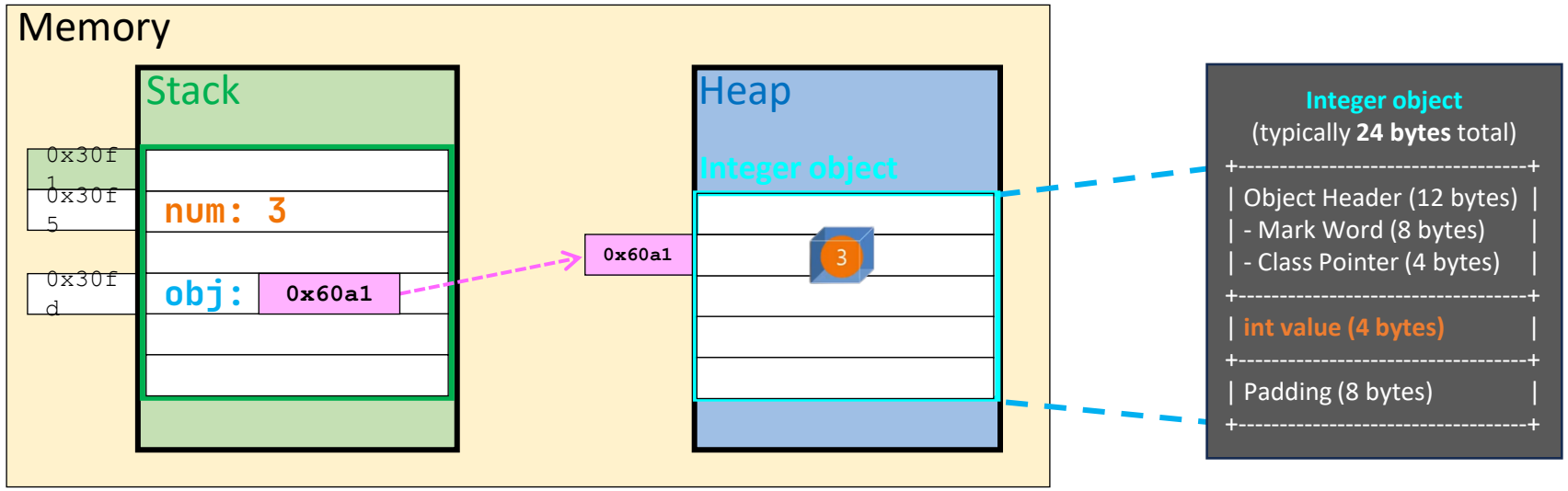
Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

```
Integer obj = 3;      // Auto-boxing   (int     --> Integer)
int     num = obj;    // Auto-unboxing (Integer --> int    )
```

```
After auto-boxing   : obj = 3
After auto-unboxing : num = ?
```



What is a Wrapper Class?

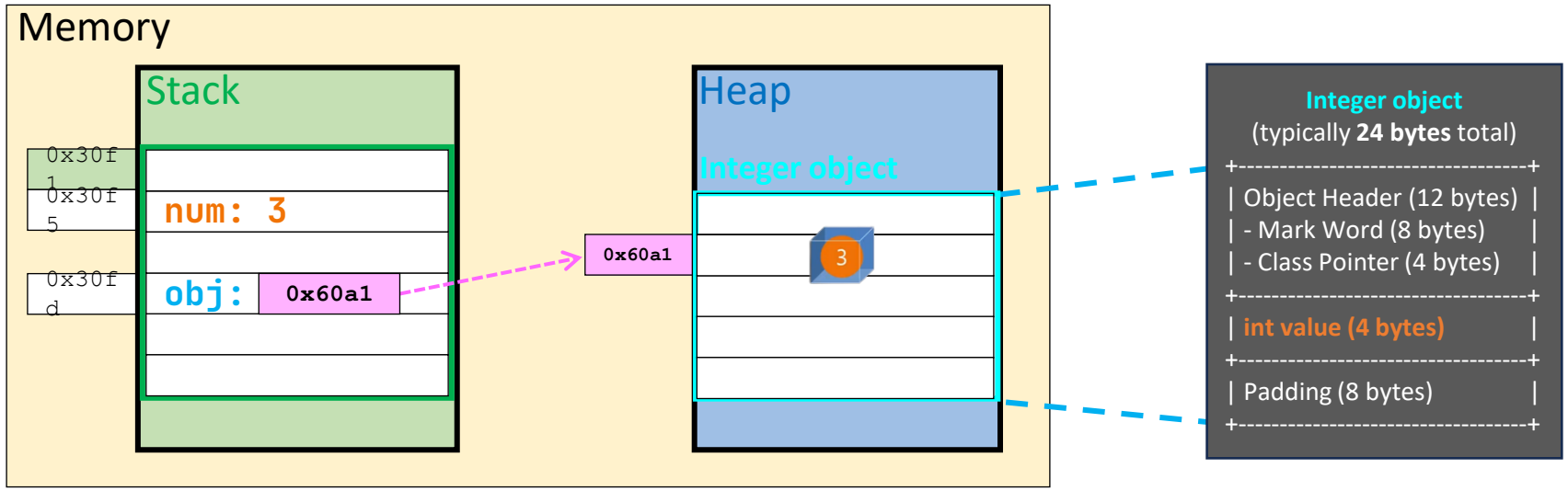
Wrapper Class : a class that "**wraps**" a primitive type into an **object**.

- converts primitive data type → **object**

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

```
Integer obj = 3;      // Auto-boxing   (int     --> Integer)
int     num = obj;    // Auto-unboxing (Integer --> int    )
```

```
After auto-boxing   : obj = 3
After auto-unboxing : num = 3
```



Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory		
Nullability		
Initialization		
Usage in Generics / Collections		
Methods		
Comparison		
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	
Nullability		
Initialization		
Usage in Generics / Collections		
Methods		
Comparison		
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored	
Nullability		
Initialization		
Usage in Generics / Collections		
Methods		
Comparison		
Caching		

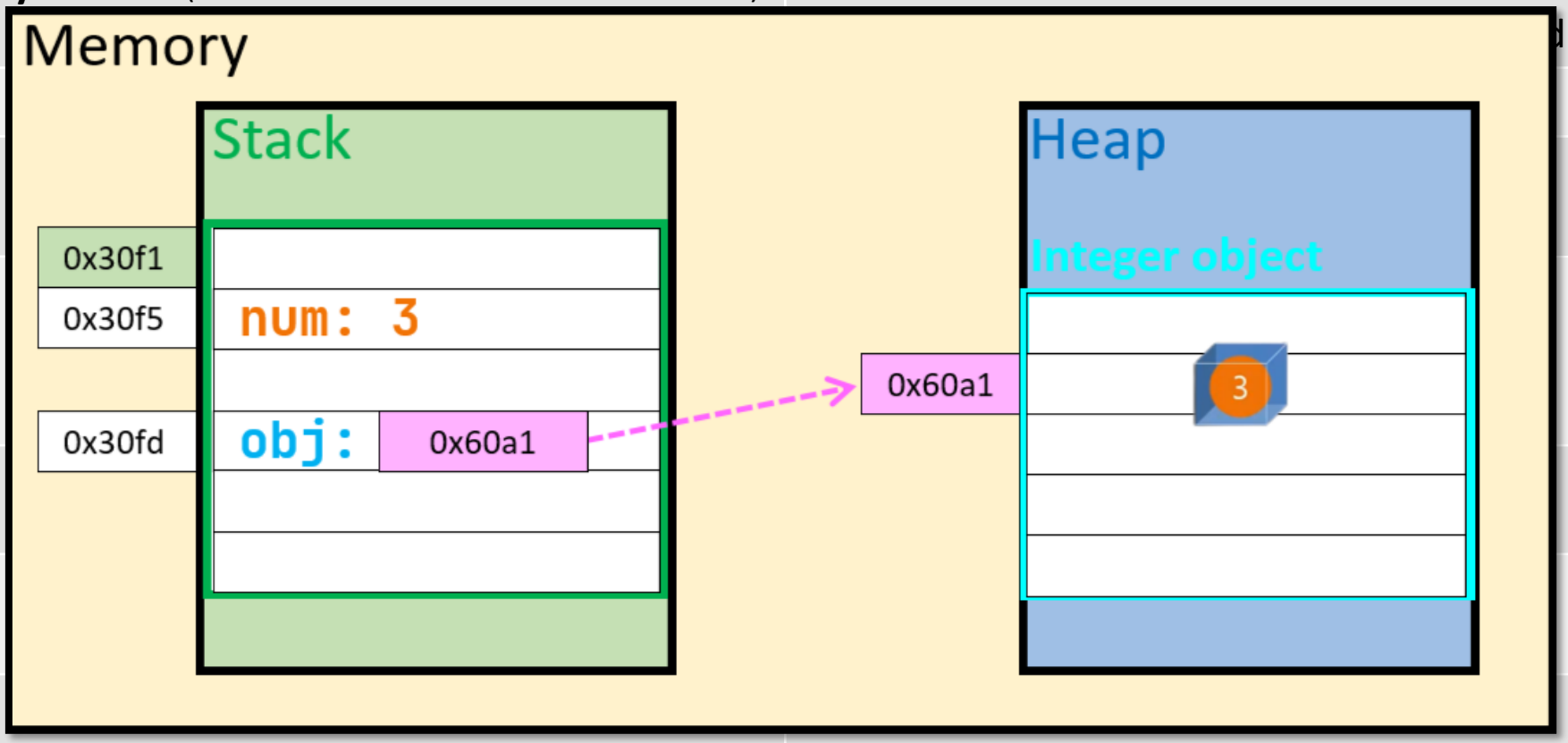
```
public class Person {
    String name;
    int ID;
    boolean isStudent;
}
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability		
Initialization		
Usage in Generics / Collections		
Methods		
Comparison		
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in stack (for **local variables** in a method call frame) stored	reference stored in stack d in heap
Nullability		
Initialization		
Usage in Generics / Collections		
Methods		
Comparison		
Caching		



Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization		
Usage in Generics / Collections		
Methods		
Comparison		
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	reference stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections		
Methods		
Comparison		
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !

```
public class Nullability {
    // ==== Class-level variables ====
    int             int_inst;    // instance field



    public static void main(String[] args) {
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !

```
public class Nullability {
    // ==== Class-level variables ====
    int             int_inst;    // instance field
    Integer         Int_inst;    // instance wrapper



    public static void main(String[] args) {
        Nullability obj = new Nullability();
        System.out.println(obj. int_inst);   ?
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !

```
public class Nullability {
    // ==== Class-level variables ====
    int             int_inst;    // instance field
    Integer         Int_inst;    // instance wrapper


    public static void main(String[] args) {
        Nullability obj = new Nullability();
        System.out.println(obj. int_inst);   0
        System.out.println(obj. Int_inst);   ?
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !

```
public class Nullability {
    // ==== Class-level variables ====
    int             int_inst;    // instance field
    Integer         Int_inst;    // instance wrapper


    public static void main(String[] args) {
        Nullability obj = new Nullability();
        System.out.println(obj. int_inst);   0
        System.out.println(obj. Int_inst);   null
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !

```
public class Nullability {
    // ==== Class-level variables ====
    int             int_inst;    // instance field
    Integer         Int_inst;    // instance wrapper
    static int      int_stat;    // static field
    static Integer  Int_stat;    // static wrapper

    public static void main(String[] args) {
        Nullability obj = new Nullability();
        System.out.println(obj. int_inst);   0
        System.out.println(obj. Int_inst);   null

        System.out.println(int_stat);        ?
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗

```
public class Nullability {
    // ==== Class-level variables ====
    int             int_inst;    // instance field
    Integer         Int_inst;    // instance wrapper
    static int      int_stat;    // static field
    static Integer  Int_stat;    // static wrapper

    public static void main(String[] args) {
        Nullability obj = new Nullability();
        System.out.println(obj. int_inst);   0
        System.out.println(obj. Int_inst);   null

        System.out.println(int_stat);        0
        System.out.println(Int_stat);        ?
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !

```
public class Nullability {
    // ==== Class-level variables ====
    int             int_inst;    // instance field
    Integer         Int_inst;    // instance wrapper
    static int      int_stat;    // static field
    static Integer Int_stat;     // static wrapper

    public static void main(String[] args) {
        Nullability obj = new Nullability();
        System.out.println(obj. int_inst);   0
        System.out.println(obj. Int_inst);   null

        System.out.println(int_stat);        0
        System.out.println(Int_stat);        null
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !

```
public class Nullability {
    public static void main(String[] args) {
        // ==== local variables ====
        int a;
        Integer A;
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗

```
public class Nullability {
    public static void main(String[] args) {
        // ==== local variables ====
        int a;
        Integer A;
        System.out.println(a);
```

❓

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗

```java
public class Nullability {
    public static void main(String[] args) {
        // ==== local variables ====
        int a;
        Integer A;
        System.out.println(a);   ❌ Compile error: not initialized
        System.out.println(A);   ❓
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static or instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static or instance variables*** **local variables:** must be **explicitly initialized** ❗

```java
public class Nullability {
    public static void main(String[] args) {
        // ==== local variables ====
        int a;
        Integer A;
        System.out.println(a);   ❌ Compile error: not initialized
        System.out.println(A);   ❌ Compile error: not initialized
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗

```java
public class Nullability {
    public static void main(String[] args) {
        // ==== local variables ====
        int a;
        Integer A;
        System.out.println(a);   ❌ Compile error: not initialized
        System.out.println(A);   ❌ Compile error: not initialized

        // ==== Array ====
        int[]     intArray = new int[10];
        Integer[] IntArray = new Integer[10];

        System.out.println(intArray[1]);   ❓
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗

```java
public class Nullability {
    public static void main(String[] args) {
        // ==== local variables ====
        int a;
        Integer A;
        System.out.println(a);   ❌ Compile error: not initialized
        System.out.println(A);   ❌ Compile error: not initialized

        // ==== Array ====
        int[]     intArray = new int[10];
        Integer[] IntArray = new Integer[10];

        System.out.println(intArray[1]);   0
        System.out.println(IntArray[1]);   ?
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗

```java
public class Nullability {
    public static void main(String[] args) {
        // ==== local variables ====
        int a;
        Integer A;
        System.out.println(a);   ❌ Compile error: not initialized
        System.out.println(A);   ❌ Compile error: not initialized

        // ==== Array ====
        int[]     intArray = new int[10];
        Integer[] IntArray = new Integer[10];

        System.out.println(intArray[1]);    0
        System.out.println(IntArray[1]);    null
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	reference stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections		
Methods		
Comparison		
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*
Methods		
Comparison		
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*

```java
import java.util.ArrayList;
public class Nullability {
    public static void main(String[] args) {
        // ==== Collections Framework ====
        ArrayList<int>     list1 = new ArrayList<>(10);
        ArrayList<Integer> list2 = new ArrayList<>(10);
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*

```
import java.util.ArrayList;
public class Nullability {
    public static void main(String[] args) {
        // ==== Collections Framework ====
        ArrayList<int>     list1 = new ArrayList<>(10);
        ArrayList<Integer> list2 = new ArrayList<>(10);
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*

```
import java.util.ArrayList;
public class Nullability {
    public static void main(String[] args) {
        // ==== Collections Framework ====
        ArrayList<int>     list1 = new ArrayList<>(10);
        ArrayList<Integer> list2 = new ArrayList<>(10);

        System.out.println(list2.get(0));
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*

```
import java.util.ArrayList;
public class Nullability {
    public static void main(String[] args) {
        // ==== Collections Framework ====
        ArrayList<int>     list1 = new ArrayList<>(10);
        ArrayList<Integer> list2 = new ArrayList<>(10);

        System.out.println(list2.get(0));   ❌ Error: IndexOutOfBoundsException
        System.out.println(list2.size());   ❓
```

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** ❗
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*

```java
import java.util.ArrayList;
public class Nullability {
    public static void main(String[] args) {
        // ==== Collections Framework ====
        ArrayList<int>     list1 = new ArrayList<>(10);
        ArrayList<Integer> list2 = new ArrayList<>(10);

        System.out.println(list2.get(0));   ❌ Error: IndexOutOfBoundsException
        System.out.println(list2.size());    0
```

creates a list with capacity for 10 elements
but **no elements exist** until added

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static or instance variables*** **local variables:** must be **explicitly initialized** ❗	defaults to **null:** ***static or instance variables*** **local variables:** must be **explicitly initialized** ❗
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*

```
import java.util.ArrayList;
public class Nullability {
    public static void main(String[] args) {
        // ==== Collections Framework ====
        ArrayList<int>     list1 = new ArrayList<>(10);
        ArrayList<Integer> list2 = new ArrayList<>(10);

        System.out.println(list2.get(0));   ❌ Error: IndexOutOfBoundsException
        System.out.println(list2.size());    0
```

creates a list with capacity for 10 elements but **no elements exist** until added

the list is **empty** → there's no element to retrieve

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static or instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*

```
import java.util.ArrayList;
public class Nullability {
    public static void main(String[] args) {
        // ==== Collections Framework ====
        ArrayList<Int>     list1 = new ArrayList<>(10);
        ArrayList<Integer> list2 = new ArrayList<>(10);

        System.out.println(list2.get(0));   ❌ Error: IndexOutOfBoundsException
        System.out.println(list2.size());   0
```

creates a list with capacity for 10 elements but **no elements exist** until added

the list is **empty** → there's no element to retrieve

confirming the list is **empty**

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*
Methods		
Comparison		
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*
Methods	n/a	has utility **methods** such as *parseInt(), valueOf(), intValue()*
Comparison		
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*
Methods	n/a	has utility **methods** such as *parseInt(), valueOf(), intValue()*
Comparison	**==** compares **actual values**	***.equals()*** compares **values (contents)** **==** compares object **references (identity)**
Caching		

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*
Methods	n/a	has utility **methods** such as *parseInt(), valueOf(), intValue()*
Comparison	**==** compares **actual values**	***.equals()*** compares **values (contents)** **==** compares object **references (identity)**
Caching	n/a	values between **-128 and 127** are cached for better performance and memory efficiency

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)

```
//Within cache range (-128 to 127 by default)
Integer a1 = 127;
Integer a2 = 127;



//Outside cache range
```

Caching	n/a	values between **-128 and 127** are cached for better performance and memory efficiency

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)

```
//Within cache range (-128 to 127 by default)
Integer a1 = 127;
Integer a2 = 127;
System.out.println(a1 == a2);


//Outside cache range
```

Caching	n/a	values between **-128 and 127** are cached for better performance and memory efficiency

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)

```
//Within cache range (-128 to 127 by default)
Integer a1 = 127;
Integer a2 = 127;
System.out.println(a1 == a2);          true
System.out.println(a1.equals(a2));      ?

//Outside cache range
```

Caching	n/a	values between **-128 and 127** are cached for better performance and memory efficiency

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)

```
//Within cache range (-128 to 127 by default)
Integer a1 = 127;
Integer a2 = 127;
System.out.println(a1 == a2);          true
System.out.println(a1.equals(a2));     true

//Outside cache range
Integer b1 = 128;
Integer b2 = 128;
System.out.println(b1 == b2);          ?
```

Caching	n/a	values between **-128 and 127** are cached for better performance and memory efficiency

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)

```
//Within cache range (-128 to 127 by default)
Integer a1 = 127;
Integer a2 = 127;
System.out.println(a1 == a2);          true
System.out.println(a1.equals(a2));     true

//Outside cache range
Integer b1 = 128;
Integer b2 = 128;
System.out.println(b1 == b2);          false
System.out.println(b1.equals(b2));      ?
```

Caching	n/a	values between **-128 and 127** are cached for better performance and memory efficiency

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)

```
//Within cache range (-128 to 127 by default)
Integer a1 = 127;
Integer a2 = 127;
System.out.println(a1 == a2);          true
System.out.println(a1.equals(a2));     true

//Outside cache range
Integer b1 = 128;
Integer b2 = 128;
System.out.println(b1 == b2);          false
System.out.println(b1.equals(b2));     true
```

Caching	n/a	values between **-128 and 127** are cached for better performance and memory efficiency

Primitive vs Wrapper Class

	Primitive data type (int)	Wrapper Class (Integer)
Memory	stored **directly** in **stack** (for **local variables** in a method call frame) stored in **heap** (if part of an **object**)	**reference** stored in **stack** **object** (which holds the actual value) stored in **heap**
Nullability	cannot be null	can be **null** (points **no object** = the object does **not exist**)
Initialization	defaults to **0:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !	defaults to **null:** ***static*** *or* ***instance variables*** **local variables:** must be **explicitly initialized** !
Usage in Generics / Collections	**n/a**	required in **generics** and **collection frameworks** such as *List<**Integer**>* *ArrayList<**Double**>* *HashMap<**String**, **Integer**>*
Methods	n/a	has utility **methods** such as *parseInt(), valueOf(), intValue()*
Comparison	**==** compares **actual values**	***.equals()*** compares **values (contents)** **==** compares object **references (identity)**
Caching	n/a	values between **-128 and 127** are cached for better performance and memory efficiency

Generic Class & Method

```
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = value;
    }


}
```

Generic Class & Method

```java
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = value;
    }

    public <U> void printLable(U other) {
        System.out.println("T: " + value + ", U: " + other);
    }



}
```

Generic Class & Method

```
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = value;
    }

    public <U> void printLable(U other) {
        System.out.println("T: " + value + ", U: " + other);
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");



    }
}
```

Generic Class & Method

```java
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = value;
    }

    public <U> void printLable(U other) {
        System.out.println("T: " + value + ", U: " + other);
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");
        box1.printLable("DC");
    }
}
```

Generic Class & Method

```
public class BoxLabel<T> {
    private T "Apple"

    public BoxLabel(T value) {
        this.value = "Apple"
    }

    public <U> void printLable(U other) {
        System.out.println("T: " + value + ", U: " + other);
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");
        box1.printLable("DC");
    }
}
```

Generic Class & Method

```
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = "Apple"
    }

    public <U> void printLable(U other) {
        System.out.println("T: " +"Apple"+ ", U: " + other);
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");
        box1.printLable("DC");
    }
}
```

Generic Class & Method

```
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = "Apple"
    }

    public <U> void printLable(U other) {
        System.out.println("T: " +"Apple"+ ", U: " + "DC" );
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");
        box1.printLable("DC");
    }
}
```

Generic Class & Method

```
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = "Apple"
    }

    public <U> void printLable(U other) {
        System.out.println("T: " +"Apple"+ ", U: " + "DC" );
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");
        box1.printLable("DC");      T: Apple, U: true



    }
}
```

Generic Class & Method

```java
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = value;
    }

    public <U> void printLable(U other) {
        System.out.println("T: " + value + ", U: " + other);
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");
        box1.printLable("DC");       T: Apple, U: true
        box1.printLable(20500);  ?
    }
}
```

Generic Class & Method

```java
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = value;
    }

    public <U> void printLable(U other) {
        System.out.println("T: " + value + ", U: " + other);
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");
        box1.printLable("DC");      T: Apple, U: true
        box1.printLable(20500);     T: Apple, U: 20500

        BoxLabel<Integer> box2 = new BoxLabel <>(2025);
        box2.printLable("Apple");

    }
}
```

Generic Class & Method

```java
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = value;
    }

    public <U> void printLable(U other) {
        System.out.println("T: " + value + ", U: " + other);
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");
        box1.printLable("DC");      T: Apple, U: true
        box1.printLable(20500);     T: Apple, U: 20500

        BoxLabel<Integer> box2 = new BoxLabel <>(2025);
        box2.printLable("Apple");   T: 2025, U: Apple
        box2.printLable(20.5);    ?
    }
}
```

Generic Class & Method

```java
public class BoxLabel<T> {
    private T value;

    public BoxLabel(T value) {
        this.value = value;
    }

    public <U> void printLable(U other) {
        System.out.println("T: " + value + ", U: " + other);
    }

    public static void main(String[] args) {
        BoxLabel<String> box1 = new BoxLabel <>("Apple");
        box1.printLable("DC");      T: Apple, U: true
        box1.printLable(20500);     T: Apple, U: 20500

        BoxLabel<Integer> box2 = new BoxLabel <>(2025);
        box2.printLable("Apple");   T: 2025, U: Apple
        box2.printLable(20.5);      T: 2025, U: 20.5
    }
}
```

Why Generics?

- **R**eliability

  Error Detection at **Compile Time**: prevents Runtime Errors

- **R**eadability

- **R**eusability

Why Generics?

- **R**eliability

  Error Detection at **Compile Time**: prevents Runtime Errors

- **R**eadability

  Enhances Code Clarity:

  by Specifying Allowable Types for Objects

- **R**eusability

Why Generics?

- **R**eliability

  Error Detection at **Compile Time**: prevents Runtime Errors

- **R**eadability

  Enhances Code Clarity:

  by Specifying Allowable Types for Objects

- **R**eusability

  Enables Creation of Generic Methods and Classes:

  for various data types

Generics

A way to **define**

class **interface** **method**

once — then **reuse** with

Generics

A way to **define**

class **interface** **method**

once — then **reuse** with

many different data types

in a type safe way

Generics

Type

Name

=

Value

Generics

Type

Box manual:
“What kind of box is it?”
“What’s allowed inside this box?”

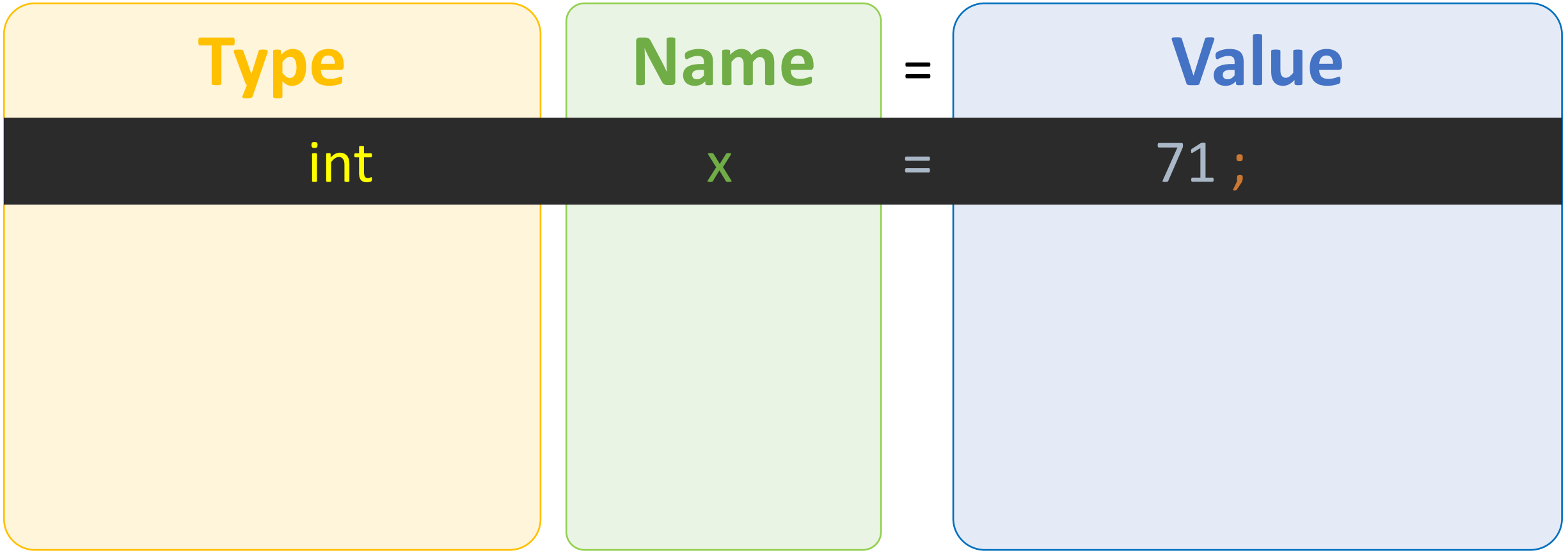
Name

Name tag:
“What do we call this box?”

=

Value

Generics



Generics

Type	Name	=	Value
int	x	=	71 ;
String[ ]	car	=	new String[100];

Generics

Type	Name	=	Value
int	x	=	71 ;
String[ ]	car	=	new String[100];
ArrayList <String>	book	=	new ArrayList<>();

Generics

Type	Name	=	Value
int	x	=	71 ;
String[ ]	car	=	new String[100];
ArrayList <String>	book	=	new ArrayList<>();
HashMap <String,Byte>	hmap	=	new HashMap<>() ;

Generics

```
public class Box <T> { }
public interface Fly <T1, T2> { }
public <T> void print (T paramName) { }
```

Generics

```
public class Box <T> { }
public interface Fly <T1, T2> { }
public <T> void print (T paramName) { }

public   ArrayList   <String> list;
private HashMap  <String, Integer> map;
public   LinkedList <Double> prices;
```

Generics: Class-level vs Method-level

```
1  class Pair<T> {
2      T var1,var2;
3      Pair(T var1, T var2) {
4          this.var1 = var1;
5          this.var2 = var2;
6      }
7
8
9
10
11
12
13
14
15 }
```

Generics: Class-level vs Method-level

```
1  class Pair<T> {
2      T var1,var2;
3      Pair(T var1, T var2) {
4          this.var1 = var1;
5          this.var2 = var2;
6      }
7      public void mT(T var3) {
8         System.out.println(var1 + " & " + var2 + ": " + var3);
9         System.out.println("Same type? " + (var1.getClass() == var3.getClass()));     }
10
11
12
13
14
15 }
```

Generics: Class-level vs Method-level

```
1  class Pair<T> {
2      T var1,var2;
3      Pair(T var1, T var2) {
4          this.var1 = var1;
5          this.var2 = var2;
6      }
7      public void mT(T var3) {
8         System.out.println(var1 + " & " + var2 + ": " + var3);
9         System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }
10
11     //generic method
12     public <S> void mS(S var3) {
13         System.out.println(var1 + " & " + var2 + ": " + var3);
14         System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }
15 }
```

Generics: Class-level vs Method-level

```
1  class Pair<T> {
2      T var1,var2;
3      Pair(T var1, T var2) {
4          this.var1 = var1;
5          this.var2 = var2;
6      }
7      public void mT(T var3) {
8         System.out.println(var1 + " & " + var2 + ": " + var3);
9         System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }
10
11     //generic method
12     public <S> void mS(S var3) {
13         System.out.println(var1 + " & " + var2 + ": " + var3);
14         System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }
15 }
```

```
Pair<String> p = new Pair<>("Apple", "Banana");
```

Generics: Class-level vs Method-level

```java
class Pair<T> {
    T var1,var2;
    Pair(T var1, T var2) {
        this.var1 = var1;
        this.var2 = var2;
    }
    public void mT(T var3) {
       System.out.println(var1 + " & " + var2 + ": " + var3);
       System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }

    //generic method
    public <S> void mS(S var3) {
        System.out.println(var1 + " & " + var2 + ": " + var3);
        System.out.println("Same type? " + (var1.getClass() == var3.getClass()));   }
}
```

```java
Pair<String> p = new Pair<>("Apple", "Banana");
p.mT("Cherry");   // T: String
```

Apple & Banana: Cherry

Generics: Class-level vs Method-level

```java
class Pair<T> {
    T var1,var2;
    Pair(T var1, T var2) {
        this.var1 = var1;
        this.var2 = var2;
    }
    public void mT(T var3) {
       System.out.println(var1 + " & " + var2 + ": " + var3);
       System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }

    //generic method
    public <S> void mS(S var3) {
        System.out.println(var1 + " & " + var2 + ": " + var3);
        System.out.println("Same type? " + (var1.getClass() == var3.getClass()));   }
}
```

```java
Pair<String> p = new Pair<>("Apple", "Banana");
p.mT("Cherry");   // T: String
p.mT(100);        // T: String
```

Apple & Banana: Cherry

Generics: Class-level vs Method-level

```java
class Pair<T> {
    T var1,var2;
    Pair(T var1, T var2) {
        this.var1 = var1;
        this.var2 = var2;
    }
    public void mT(T var3) {
       System.out.println(var1 + " & " + var2 + ": " + var3);
       System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }

    //generic method
    public <S> void mS(S var3) {
        System.out.println(var1 + " & " + var2 + ": " + var3);
        System.out.println("Same type? " + (var1.getClass() == var3.getClass()));   }
}
```

```java
Pair<String> p = new Pair<>("Apple", "Banana");
p.mT("Cherry");    // T: String
p.mT(100);         // T: String
```

Apple & Banana: Cherry

Generics: Class-level vs Method-level

```
1  class Pair<T> {
2      T var1,var2;
3      Pair(T var1, T var2) {
4          this.var1 = var1;
5          this.var2 = var2;
6      }
7      public void mT(T var3) {
8         System.out.println(var1 + " & " + var2 + ": " + var3);
9         System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }
10
11     //generic method
12     public <S> void mS(S var3) {
13         System.out.println(var1 + " & " + var2 + ": " + var3);
14         System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }
15 }
```

```
Pair<String> p = new Pair<>("Apple", "Banana");
```

Generics: Class-level vs Method-level

```java
class Pair<T> {
    T var1,var2;
    Pair(T var1, T var2) {
        this.var1 = var1;
        this.var2 = var2;
    }
    public void mT(T var3) {
       System.out.println(var1 + " & " + var2 + ": " + var3);
       System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }

    //generic method
    public <S> void mS(S var3) {
        System.out.println(var1 + " & " + var2 + ": " + var3);
        System.out.println("Same type? " + (var1.getClass() == var3.getClass()));   }
}
```

```java
Pair<String> p = new Pair<>("Apple", "Banana");
p.mS("Cherry");   // U:String
```

Apple & Banana: Cherry

Generics: Class-level vs Method-level

```java
class Pair<T> {
    T var1,var2;
    Pair(T var1, T var2) {
        this.var1 = var1;
        this.var2 = var2;
    }
    public void mT(T var3) {
       System.out.println(var1 + " & " + var2 + ": " + var3);
       System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }

    //generic method
    public <S> void mS(S var3) {
        System.out.println(var1 + " & " + var2 + ": " + var3);
        System.out.println("Same type? " + (var1.getClass() == var3.getClass()));   }
}
```

```java
Pair<String> p = new Pair<>("Apple", "Banana");
p.mS("Cherry");   // U:String
p.mS(100);        // U:Integer
```

Apple & Banana: Cherry

Generics: Class-level vs Method-level

```java
class Pair<T> {
    T var1,var2;
    Pair(T var1, T var2) {
        this.var1 = var1;
        this.var2 = var2;
    }
    public void mT(T var3) {
       System.out.println(var1 + " & " + var2 + ": " + var3);
       System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }

    //generic method
    public <S> void mS(S var3) {
        System.out.println(var1 + " & " + var2 + ": " + var3);
        System.out.println("Same type? " + (var1.getClass() == var3.getClass()));   }
}
```

```java
Pair<String> p = new Pair<>("Apple", "Banana");
p.mS("Cherry");    // U:String
p.mS(100);         // U:Integer
```

Apple & Banana: Cherry
Same type? **false**

Generics: Class-level vs Method-level

```java
class Pair<T> {
    T var1,var2;
    Pair(T var1, T var2) {
        this.var1 = var1;
        this.var2 = var2;
    }
    public void mT(T var3) {
       System.out.println(var1 + " & " + var2 + ": " + var3);
       System.out.println("Same type? " + (var1.getClass() == var3.getClass()));    }

    //generic method
    public <S> void mS(S var3) {
        System.out.println(var1 + " & " + var2 + ": " + var3);
        System.out.println("Same type? " + (var1.getClass() == var3.getClass()));   }
}
```

```java
Pair<String> p = new Pair<>("Apple", "Banana");
p.mS("Cherry");    // U:String
p.mS(100);         // U:Integer
p.mS(3.14);        // U:Double
```

Apple & Banana: Cherry
Same type? **false**
Same type? **false**

Generics: Upper-Bounded

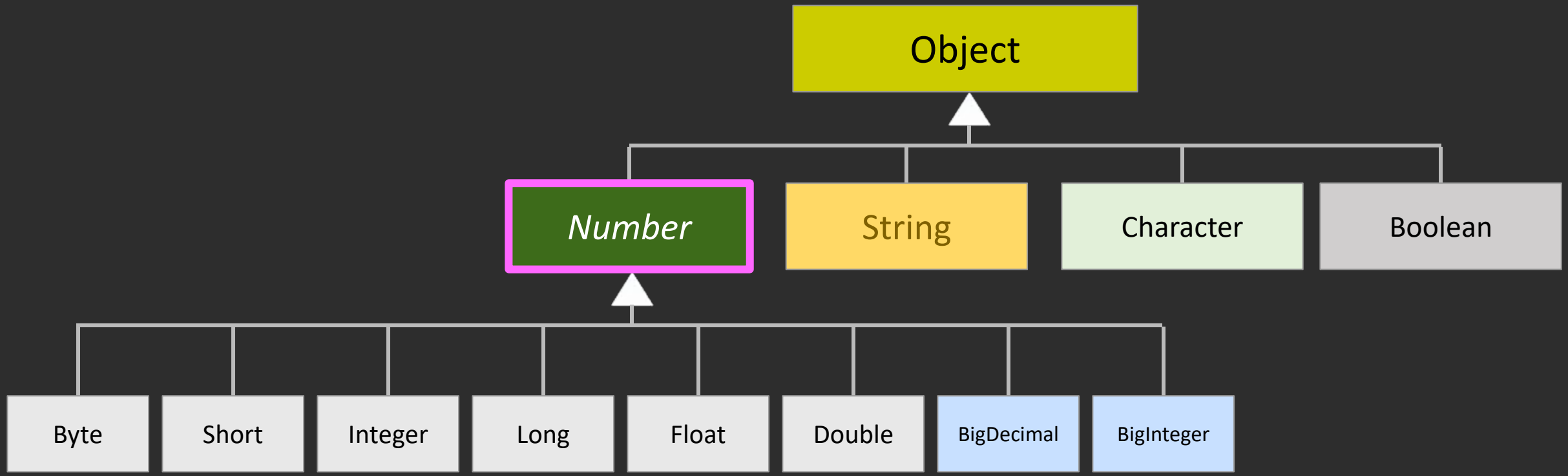
Generics: Upper-Bounded

```
p.cal(42, 100)          // Integer
p.cal(3.14, 9.99)       // Double
p.cal(100L, 200L)       // Long
```

Generics: Upper-Bounded

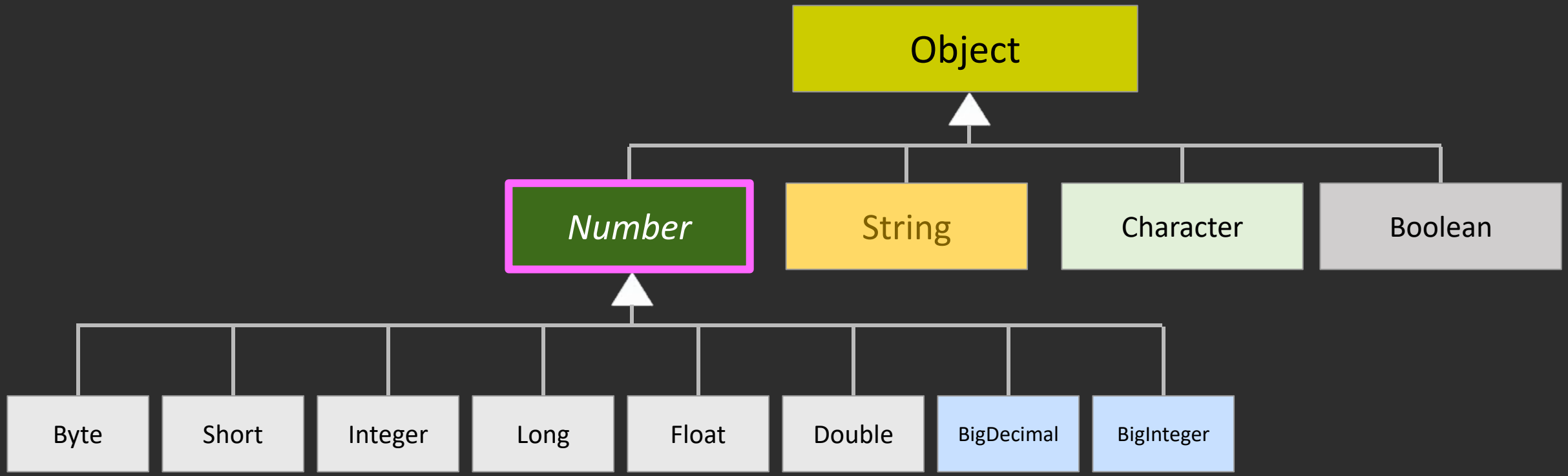
```
p.cal(42, 100)          // Integer
p.cal(3.14, 9.99)       // Double
p.cal(100L, 200L)       // Long
p.cal("anything")       // ERROR
p.cal(true)             // ERROR
```

Generics: Upper-Bounded



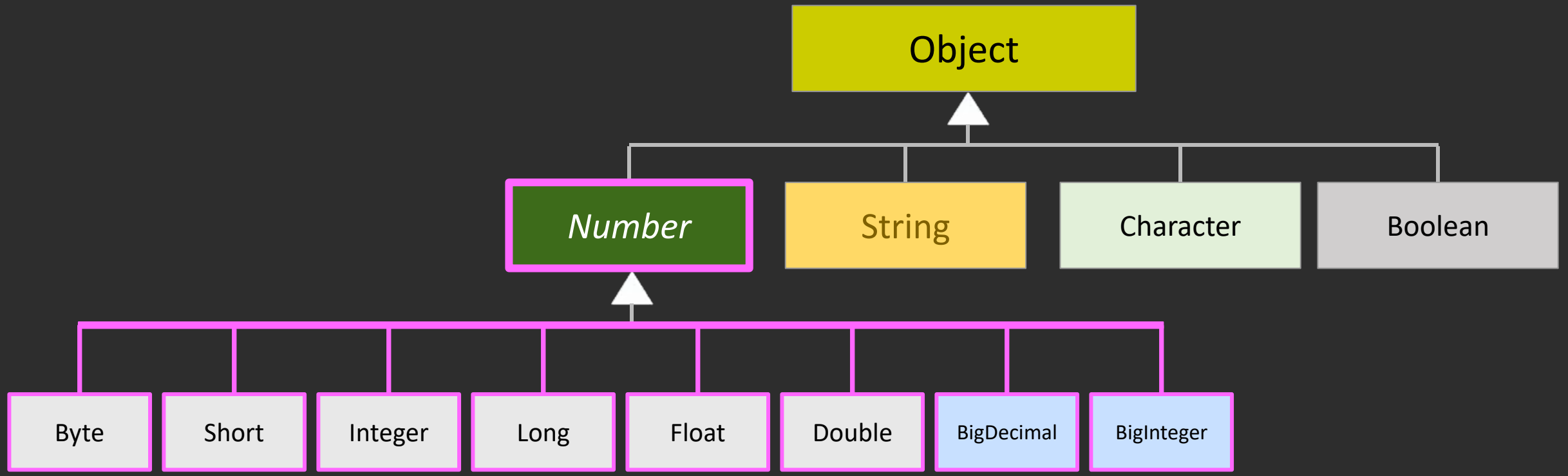

```
p.cal(42, 100)          // Integer
p.cal(3.14, 9.99)       // Double
p.cal(100L, 200L)       // Long
p.cal("anything")       // ERROR
p.cal(true)             // ERROR
```

Generics: Upper-Bounded




```
p.cal(42, 100)          // Integer
p.cal(3.14, 9.99)       // Double
p.cal(100L, 200L)       // Long
p.cal("anything")       // ERROR
p.cal(true)             // ERROR
```

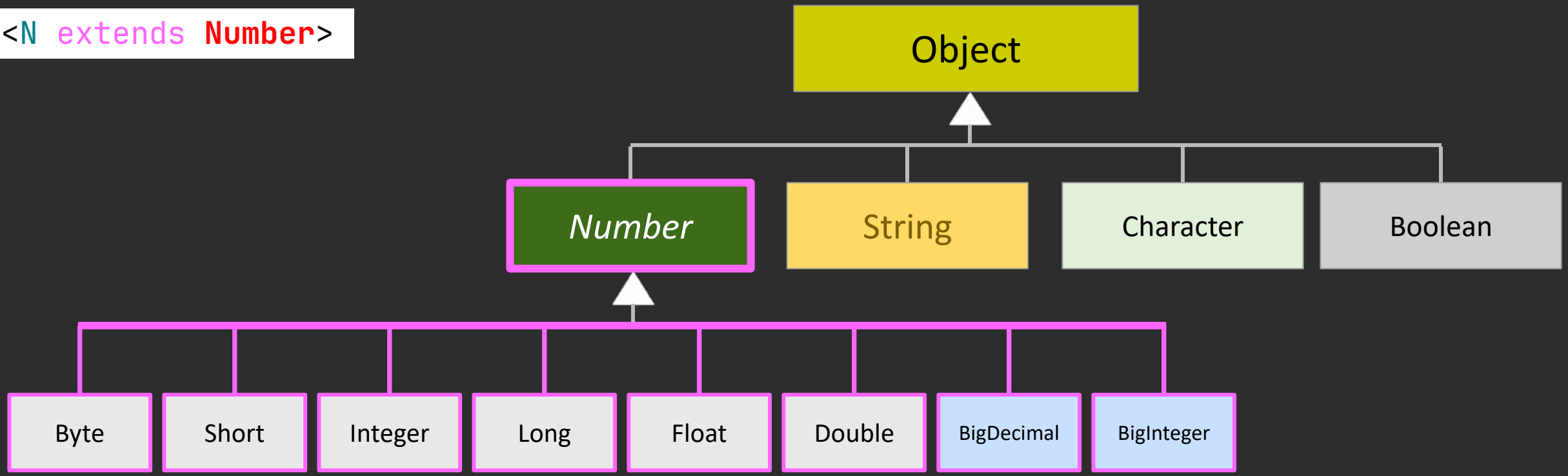
Generics: Upper-Bounded




```
p.cal(42, 100)          // Integer
p.cal(3.14, 9.99)       // Double
p.cal(100L, 200L)       // Long
p.cal("anything")       // ERROR
p.cal(true)             // ERROR
```

Generics: Upper-Bounded

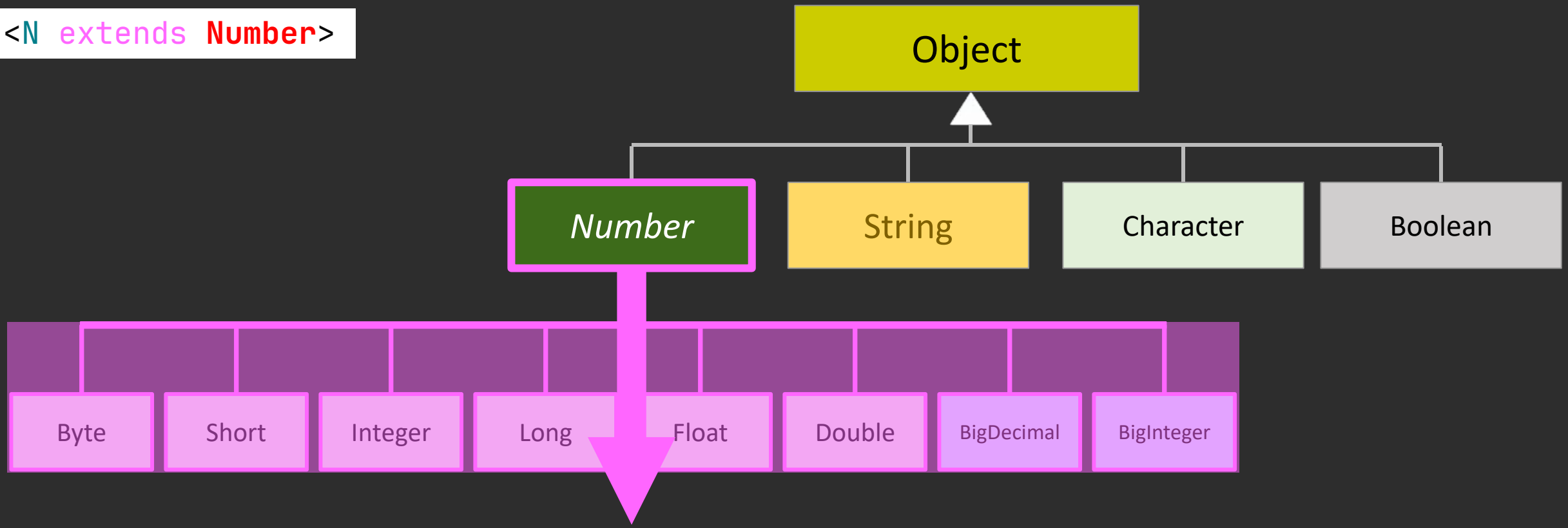
```
<N extends Number>
```




```
p.cal(42, 100)          // Integer
p.cal(3.14, 9.99)       // Double
p.cal(100L, 200L)       // Long
p.cal("anything")       // ERROR
p.cal(true)             // ERROR
```

Generics: Upper-Bounded

```
<N extends Number>
```




```
p.cal(42, 100)          // Integer
p.cal(3.14, 9.99)       // Double
p.cal(100L, 200L)       // Long
p.cal("anything")       // ERROR
p.cal(true)             // ERROR
```

Generics: Upper-Bounded

```java
class Pair<T> {
    // Upper-bounded generic method
    public <N extends Number> void print(N[] arr) {
        for (N num : arr) {
            System.out.print(num + " ");
        }
    }
}

//Test code after instantiating p
Integer[] intArr = {1, 2, 3};
Double[]  dblArr = {1.1, 2.2, 3.3};
Long[]    lngArr = {100L, 200L, 300L};
String[]  strArr = {"hi", "bye"};
```

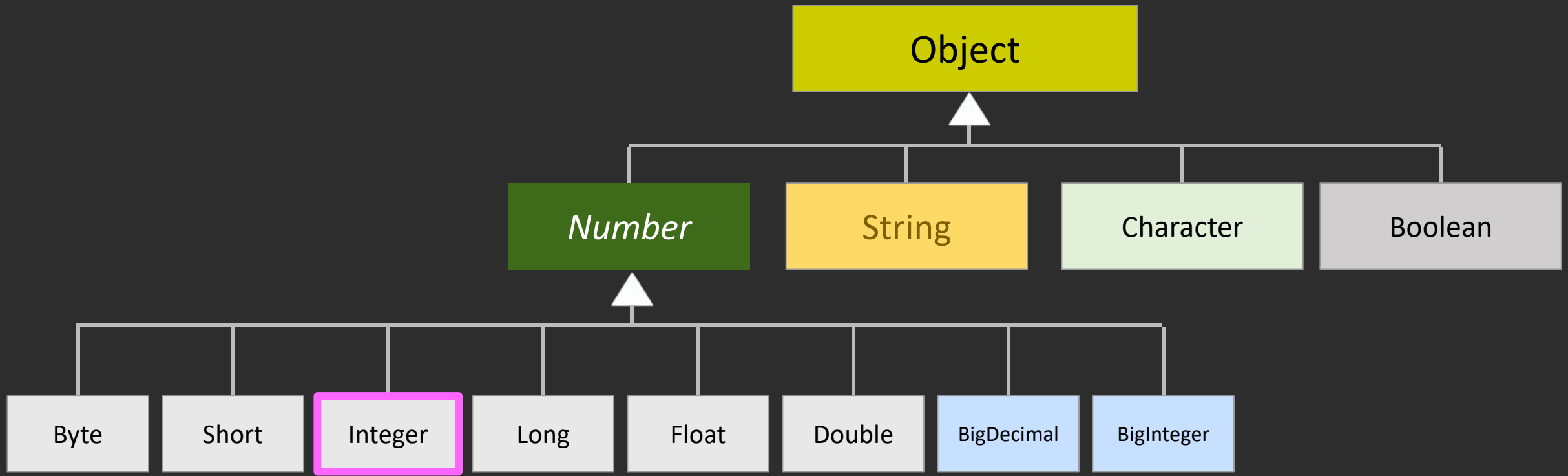
Generics: Upper-Bounded

```java
class Pair<T> {
    // Upper-bounded generic method
    public <N extends Number> void print(N[] arr) {
        for (N num : arr) {
            System.out.print(num + " ");
        }
    }
}

//Test code after instantiating p
Integer[] intArr = {1, 2, 3};
Double[]  dblArr = {1.1, 2.2, 3.3};
Long[]    lngArr = {100L, 200L, 300L};
String[]  strArr = {"hi", "bye"};

p.print(intArr)      // Integer
p.print(dblArr);     // Double
p.print(lngArr)      // Long
```

Generics: Upper-Bounded

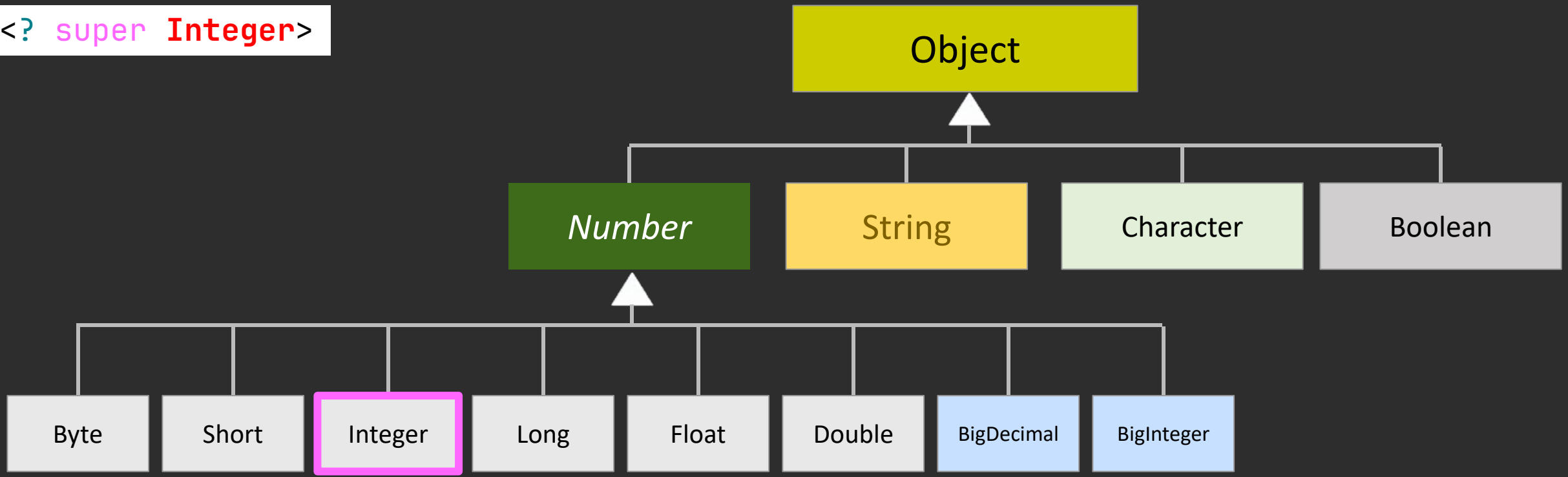
```java
class Pair<T> {
    // Upper-bounded generic method
    public <N extends Number> void print(N[] arr) {
        for (N num : arr) {
            System.out.print(num + " ");
        }
    }
}

//Test code after instantiating p
Integer[] intArr = {1, 2, 3};
Double[]  dblArr = {1.1, 2.2, 3.3};
Long[]    lngArr = {100L, 200L, 300L};
String[]  strArr = {"hi", "bye"};

p.print(intArr)      // Integer
p.print(dblArr)      // Double
p.print(lngArr)      // Long
p.print(strArr)      // String ERROR
```

Generics: Lower-Bounded

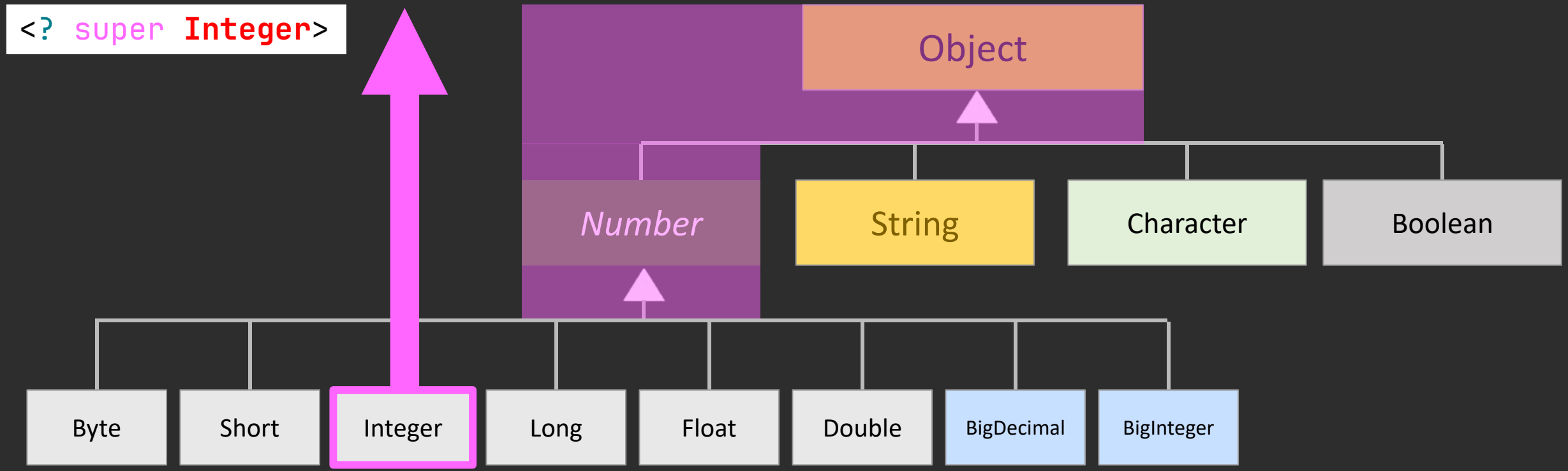


Generics: Lower-Bounded

```
<? super Integer>
```



Generics: Lower-Bounded



Generics: Lower-Bounded

```java
//Lower-bounded method
public static void stackBox(ArrayList<? super Integer> arr) {
    arr.add(1);    // ✓ Integer
    arr.add(100);  // ✓ Integer


                                                          }
```

Generics: Lower-Bounded

```java
//Lower-bounded method
public static void stackBox(ArrayList<? super Integer> arr) {
    arr.add(1);    // ✓ Integer
    arr.add(100);  // ✓ Integer
    //arr.add(3.14);  // ✗ ERROR: Double is not allowed
    //arr.add("hi");  // ✗ ERROR: String is not allowed    }
```

Generics: Lower-Bounded

```java
//Lower-bounded method
public static void stackBox(ArrayList<? super Integer> arr) {
    arr.add(1);    // ✓ Integer
    arr.add(100);  // ✓ Integer
    //arr.add(3.14);  // ✗ ERROR: Double is not allowed
    //arr.add("hi");  // ✗ ERROR: String is not allowed    }

ArrayList<Integer> intBox = new ArrayList<>();
ArrayList<Number>  numBox = new ArrayList<>();
ArrayList<Object>  objBox = new ArrayList<>();
ArrayList<Double>  dblBox = new ArrayList<>();
```

Generics: Lower-Bounded

```java
//Lower-bounded method
public static void stackBox(ArrayList<? super Integer> arr) {
    arr.add(1);    // ✓ Integer
    arr.add(100);  // ✓ Integer
    //arr.add(3.14);  // ✗ ERROR: Double is not allowed
    //arr.add("hi");  // ✗ ERROR: String is not allowed    }

ArrayList<Integer> intBox = new ArrayList<>();
ArrayList<Number>  numBox = new ArrayList<>();
ArrayList<Object>  objBox = new ArrayList<>();
ArrayList<Double>  dblBox = new ArrayList<>();

stackBox(intBox);     // ✓ Integer
stackBox(numBox);     // ✓ Number
stackBox(objBox);     // ✓ Object

```

Generics: Lower-Bounded

```java
//Lower-bounded method
public static void stackBox(ArrayList<? super Integer> arr) {
    arr.add(1);    // ✓ Integer
    arr.add(100);  // ✓ Integer
    //arr.add(3.14);  // ✗ ERROR: Double is not allowed
    //arr.add("hi");  // ✗ ERROR: String is not allowed    }

ArrayList<Integer> intBox = new ArrayList<>();
ArrayList<Number>  numBox = new ArrayList<>();
ArrayList<Object>  objBox = new ArrayList<>();
ArrayList<Double>  dblBox = new ArrayList<>();

stackBox(intBox);     // ✓ Integer
stackBox(numBox);     // ✓ Number
stackBox(objBox);     // ✓ Object
// stackBox(dblBox);  // ✗ ERROR
```

Generics: Lower-Bounded

```java
//Lower-bounded method
public static void stackBox(ArrayList<? super Integer> arr) {
    arr.add(1);    // ✓ Integer
    arr.add(100);  // ✓ Integer
    //arr.add(3.14);  // ✗ ERROR: Double is not allowed
    //arr.add("hi");  // ✗ ERROR: String is not allowed    }

ArrayList<Integer> intBox = new ArrayList<>();
ArrayList<Number>  numBox = new ArrayList<>();
ArrayList<Object>  objBox = new ArrayList<>();
ArrayList<Double>  dblBox = new ArrayList<>();

stackBox(intBox);     // ✓ Integer
stackBox(numBox);     // ✓ Number
stackBox(objBox);     // ✓ Object
// stackBox(dblBox);  // ✗ ERROR
```

```java
System.out.println(intBox);          [1, 100]
System.out.println(numBox);          [1, 100]
System.out.println(objBox);          [1, 100]
```

Generics: class activities

Create a generic print method
that can print values of different types:

7

3.14

“Hello, world!”

Generics: class activities

```java
public class Main {

    // Generic method
    public static <T> void print(T value) {
        System.out.println(value);
    }

    public static void main(String[] args) {

        print(7);                  // Integer
        print(3.14);               // Double
        print("Hello, world!"); // String
    }
}
```

Generics: class activities

```java
public class Main {

    // Generic method
    public static <T> void print(T value) {
        System.out.println(value);
    }

    public static void main(String[] args) {

        print(7);                  // Integer
        print(3.14);               // Double
        print("Hello, world!"); // String
    }
}
```

```
7
3.14
Hello, world!
```

Generics: class activities

```java
public class Main {

    // Generic method
    public static <T> void print(T value) {
        System.out.println(value);
    }

    public static void main(String[] args) {

        print(7);                  // Integer
        print(3.14);               // Double
        print("Hello, world!"); // String
    }
}
```

```
7
3.14
Hello, world!
```